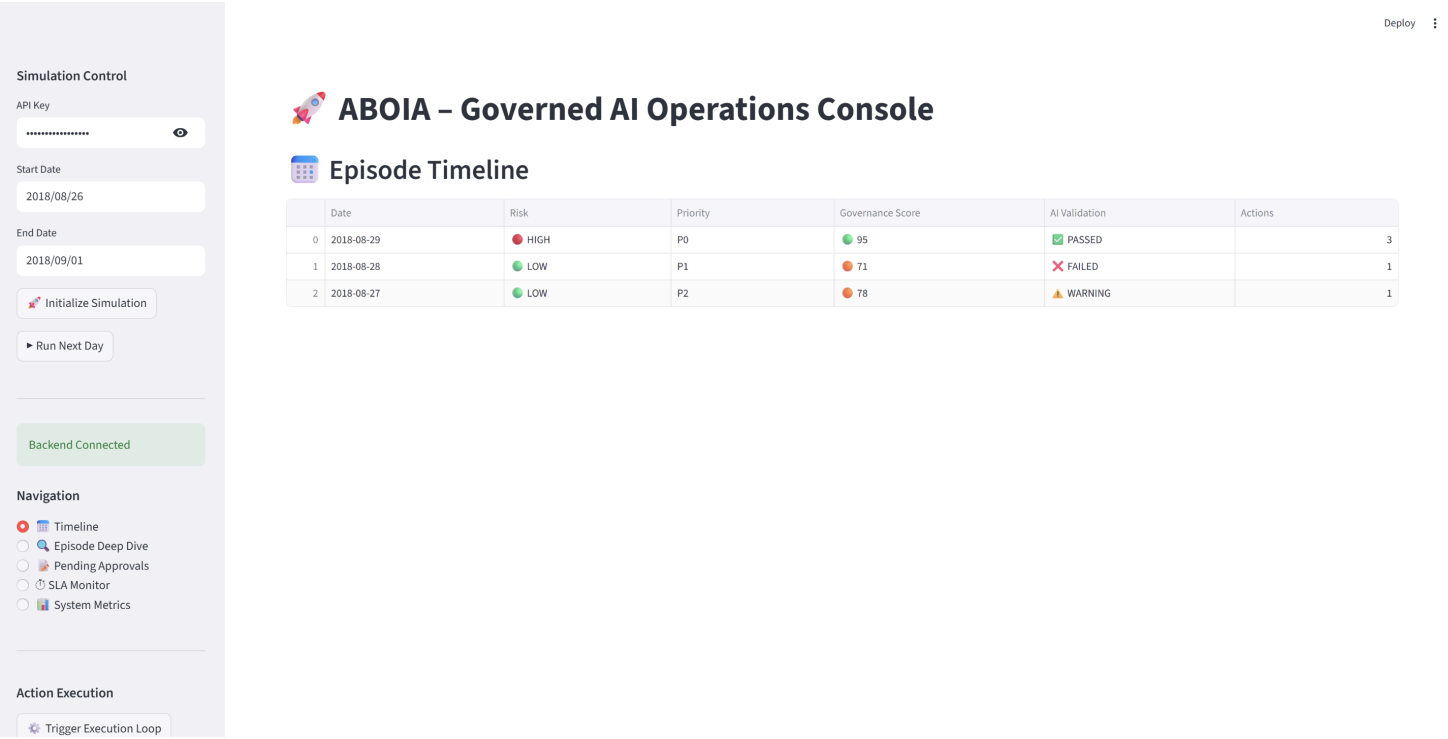
ABOIA: Autonomous Business Operations Intelligence Agent

> A governed agentic framework designed for statistical anomaly detection, LLM-powered root cause reasoning, and safe action routing — autonomously detecting business KPI anomalies, diagnosing root causes under mathematical constraints, and executing structured workflows under human-in-the-loop operator controls.



🚀 Motivation & Theoretical Background

1. Limitations of Traditional Automation

Traditional workflow systems such as Airflow DAGs or rule-based microservice pipelines are reliable for predefined workflows, but they struggle when operational patterns change unexpectedly.

For example:

- sudden traffic spikes,
- checkout failures,
- or correlated KPI drops across multiple business metrics
  often require contextual reasoning rather than fixed execution logic. In many cases, handling these scenarios requires engineers to manually update workflows or redeploy services.

LLM-based agents introduce more flexible reasoning. Given structured telemetry and anomaly summaries, an LLM can help identify likely root causes and suggest operational responses without requiring changes to hardcoded rules.

2. Risks of Unrestricted LLM Execution

While LLMs improve flexibility, allowing them to directly execute production actions introduces significant operational risk.

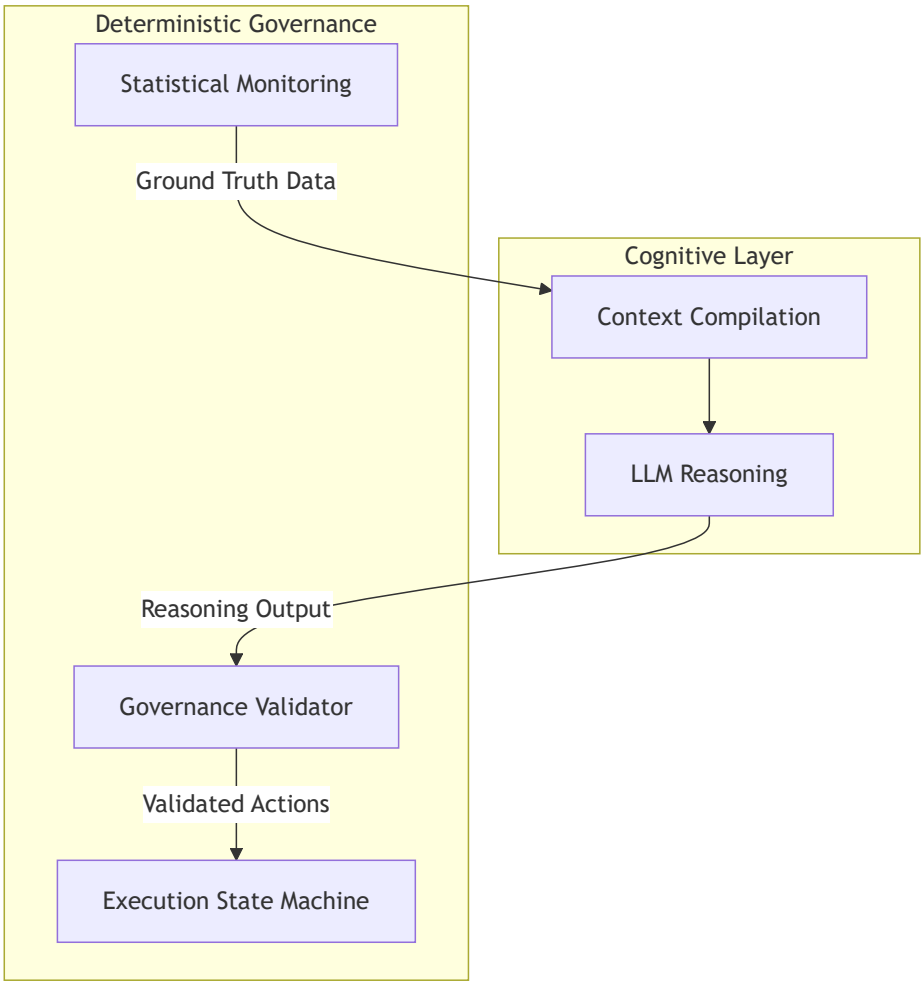
Some common failure cases include:

- **Incorrect Severity Estimation:** A minor KPI fluctuation may be interpreted as a critical infrastructure failure, resulting in unnecessary or unsafe actions.
- **Input Sensitivity:** Changes in log formatting, missing fields, or inconsistent telemetry can shift model reasoning toward incorrect conclusions.
- **Prompt Injection Risks:** Untrusted operational data may contain instructions or patterns that interfere with expected model behavior.

Because of these risks, LLM-generated reasoning should not directly control production execution workflows.

3. Dual-Track Governance Architecture

ABOIA separates reasoning and execution into two independent layers:



Cognitive Layer

The LLM is used only for reasoning and root cause analysis. It receives structured telemetry summaries and returns a constrained JSON response containing:

- root cause analysis,
- business impact,

- and risk level assessment.

The model does not directly execute actions or modify system state.

Deterministic Governance Layer

All execution logic remains deterministic and rule-based.

The monitoring engine, governance validator, and lifecycle state machine validate the LLM output before any operational action is created. Governance checks evaluate:

- metric grounding,
- reasoning quality,
- risk alignment,
- and confidence consistency.

If governance thresholds fail, the action is blocked and routed through a manual approval workflow instead of being automatically executed.

🧬 Architecture & Flow (The 5-Node Agentic Pipeline)

ABOIA is built as three independent components:

- a FastAPI backend,
- a Streamlit dashboard,
- and a SQLite persistence layer.

The core workflow is orchestrated using LangGraph.

Orchestration Model

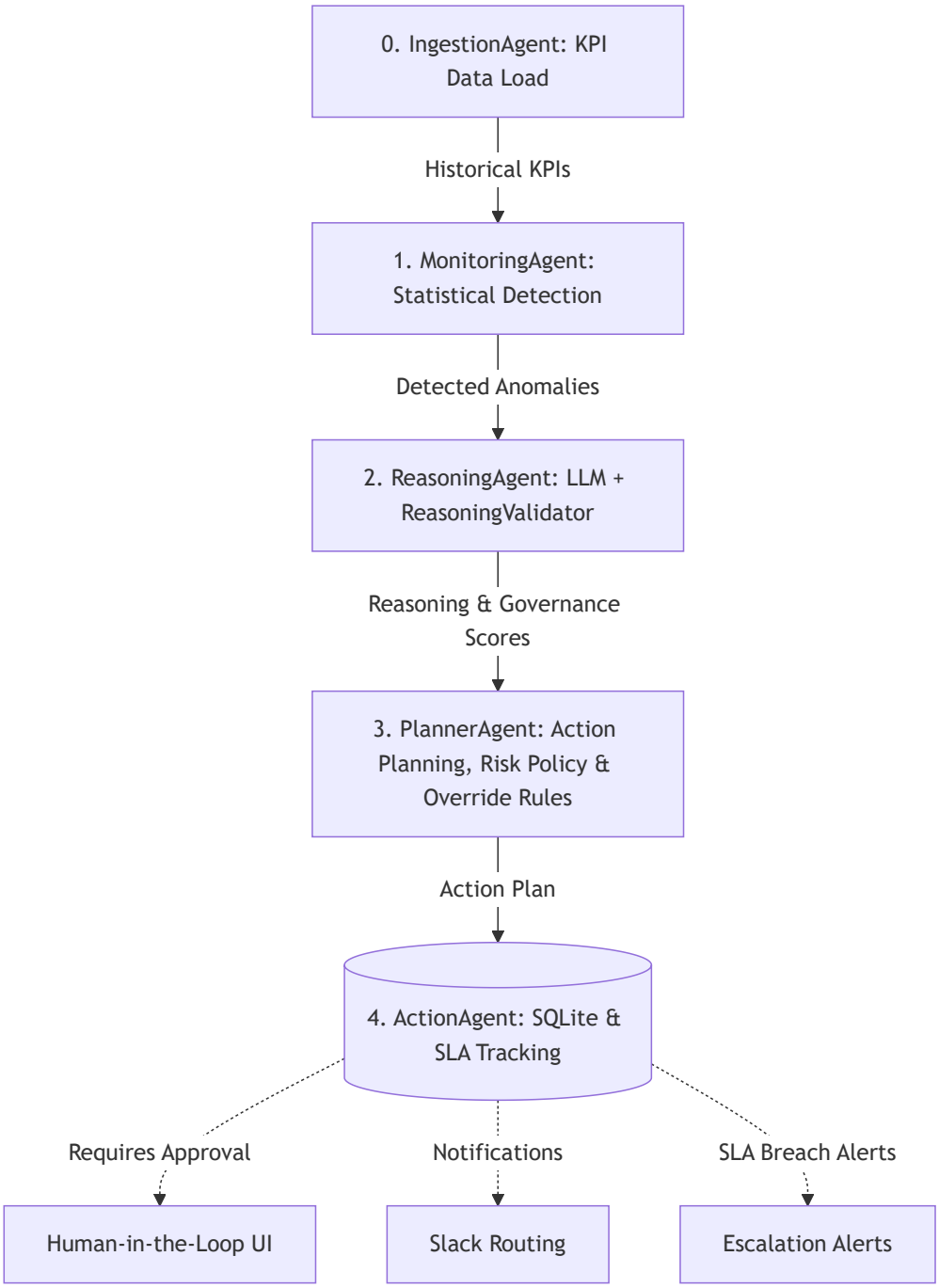
The current pipeline executes sequentially, where each agent completes before the next one begins. Although this could be implemented using chained Python functions, LangGraph was chosen to support future workflow extensions such as:

- approval-based execution interrupts,
- retry or self-correction loops,
- and more flexible stateful routing between agents.

Execution Model

The system currently runs as a daily batch simulation. Each execution processes KPIs for a single simulated date, stores the resulting state, and then advances to the next day.

This ensures chronological isolation during simulation runs, preventing agents from accessing future telemetry data.



0️⃣ Data Ingestion Engine (`IngestionAgent`)

The `IngestionAgent` loads raw transaction logs from the Olist e-commerce dataset and aggregates them into daily KPI snapshots for downstream processing.

Two ingestion safeguards are enforced:

- **Timeline Isolation:** Each simulation run only accesses data up to the current simulated date. This prevents future data leakage during anomaly detection and reasoning.
- **30-Day Warmup Window:** The first 30 days are used to build historical KPI baselines before anomaly detection begins. This reduces false positives during early initialization.

1️⃣ Statistical Monitoring Engine (`MonitoringAgent`)

Anomaly detection is handled entirely through statistical methods without involving the LLM. The `MonitoringAgent` evaluates seven KPI streams: `orders` , `unique_users` , `gmv` , `items_sold` , `aov` , `visits` , and `conversion_rate` .

The monitoring pipeline combines multiple detection strategies:

- **Rolling Z-Score & EWMA:** Detects sudden deviations and gradual metric drift.
- **Percentage Change & IQR:** Identifies sharp day-over-day changes and distribution outliers.
- **Day-of-Week Seasonality:** Compares metrics against historical values for the same weekday to account for recurring traffic patterns.
- **Cross-Metric Correlation:** Evaluates directed relationships such as `visits → orders → GMV → conversion_rate` to identify likely upstream failures without generating excessive correlated alerts.

Thresholds are continuously recalculated from historical variance data. When multiple detectors flag the same metric on the same date, the results are consolidated into a single anomaly episode before being passed to the reasoning layer.

2️⃣ Governance-Evaluated Reasoning (`ReasoningAgent` & `ReasoningValidator`)

When the `MonitoringAgent` detects an anomaly episode, the pipeline invokes the `ReasoningAgent` to generate an LLM-based diagnosis and immediately validates the result through the `ReasoningValidator` .

- **Structured Context Compilation:** Before the LLM call, anomaly data is converted into a structured summary containing KPI statistics, recent variance trends, and affected metrics. The model receives this structured payload rather than raw logs or CSV data.
- **LLM Reasoning:** Gemini returns a structured JSON response containing:
- `root_cause`
- `business_impact`
- `risk_level`

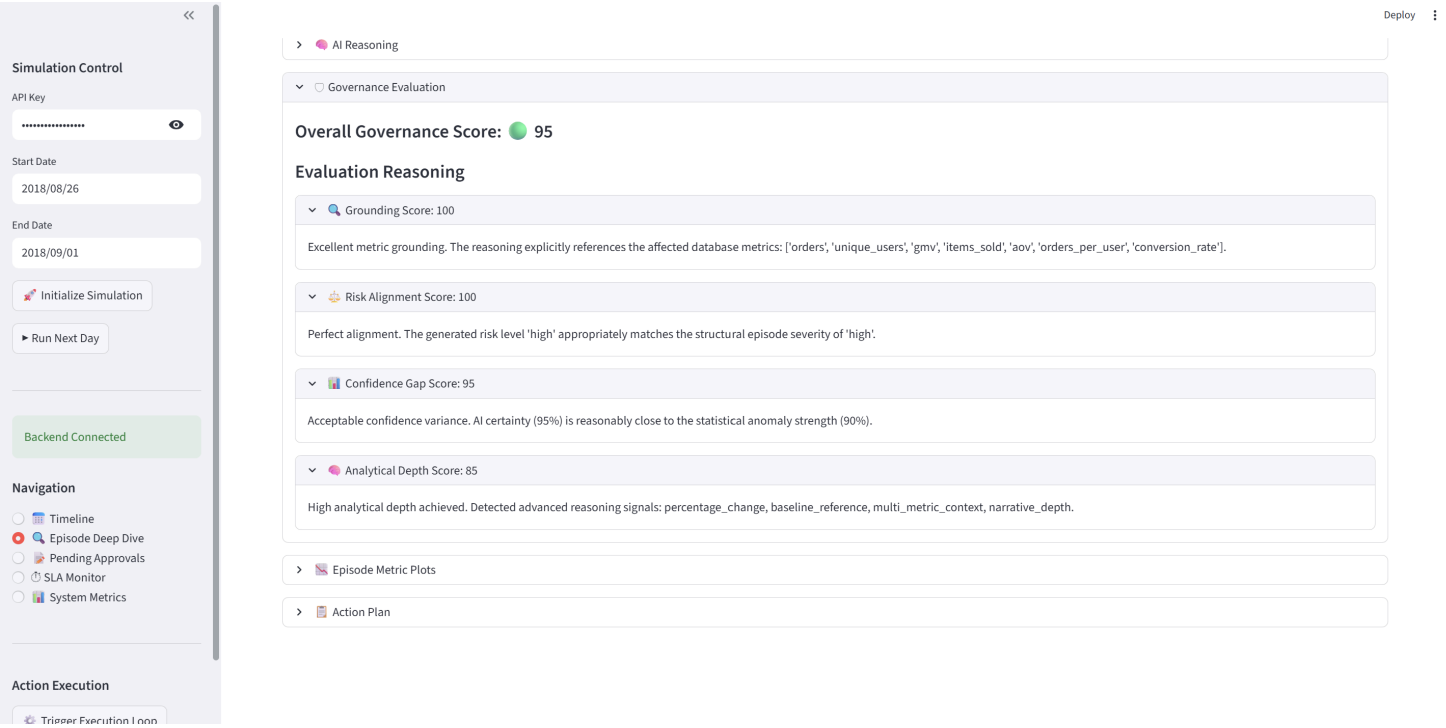
The model is restricted to reasoning only from the provided telemetry summary.

- **Governance Validation:** The `ReasoningValidator` evaluates the LLM output across four dimensions:
- **1. Metric Grounding:** Checks whether the diagnosis references the affected metrics.
- **2. Analytical Depth:** Evaluates whether the response includes meaningful statistical context and reasoning detail.
- **3. Risk Alignment:** Compares the LLM-reported risk level against the statistical severity detected by the monitoring engine.
- **4. Confidence Parity:** Measures the gap between deterministic anomaly confidence and the LLM's self-reported reasoning confidence.

The final governance score is calculated as:

$$\textbf{Overall Governance Score} = \lfloor \frac{\text{Grounding} + \text{Depth} + \text{RiskAlignment} + \text{ConfidenceParity}}{4} \rfloor$$

This score determines whether the resulting action can be executed automatically or must be routed through manual approval.



3 Governance-Aware Planning (`PlannerAgent`)

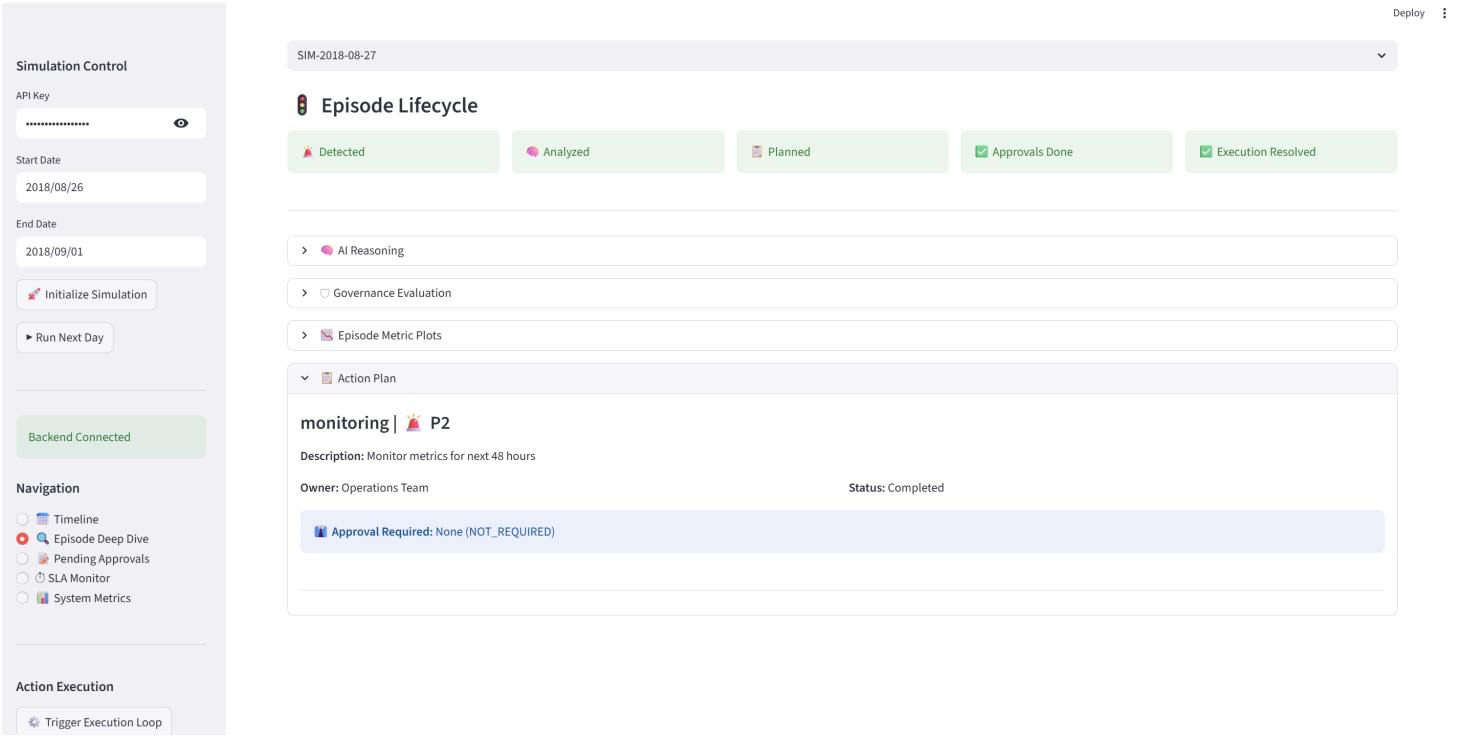
The `PlannerAgent` converts the LLM-generated diagnosis into a structured operational action.

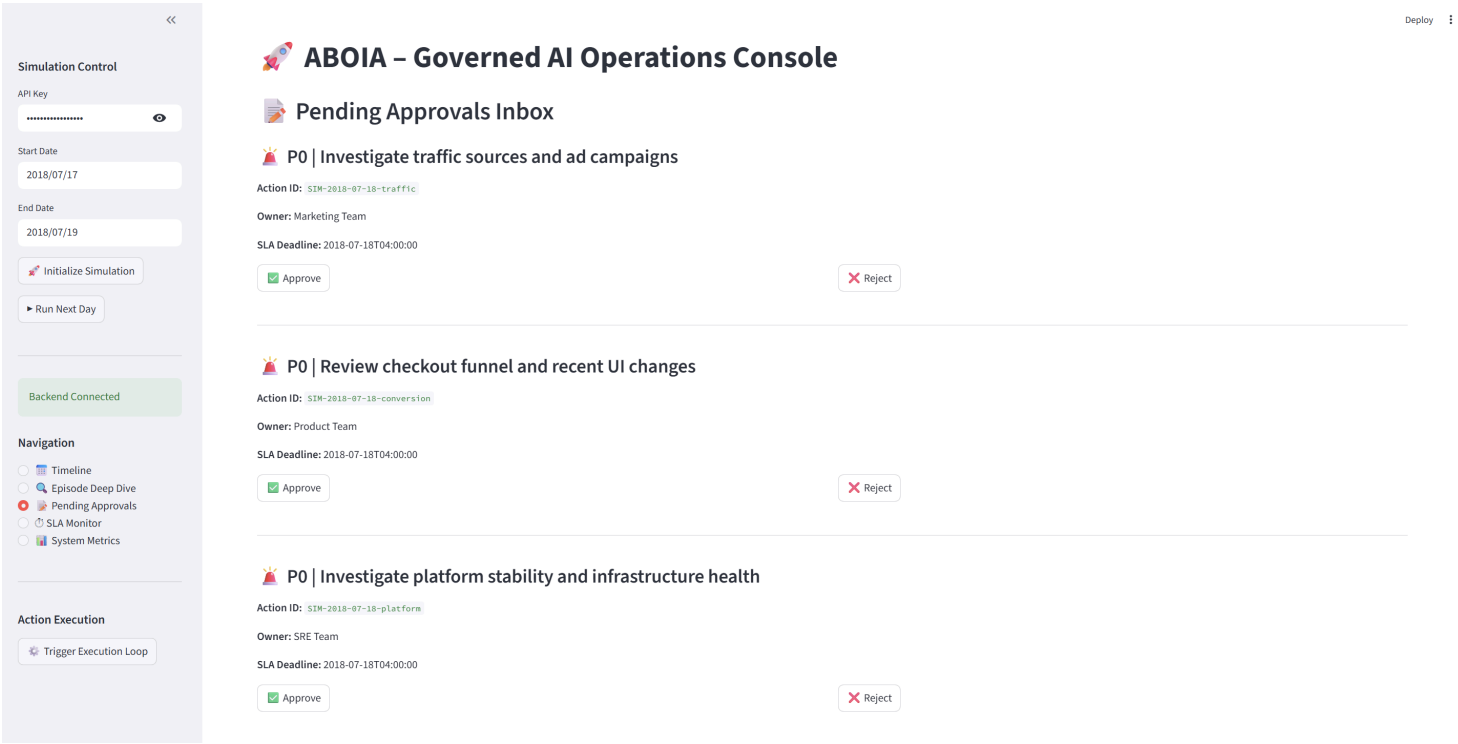
- **Keyword-to-Action Mapping:** The `root_cause` field is matched against rules defined in `root_cause_action_map.yaml` . Matching rules resolve to predefined action templates containing ticket type, owner team, priority, and SLA settings.
  If no mapping is found, the system raises an `unrecognized_root_cause_override` , escalates the ticket priority, and routes the action through manual approval.
- **Governance Override Handling:** The planner evaluates the governance scorecard returned by the `ReasoningValidator` . If any threshold is violated, the ticket priority is escalated and `requires_approval = True` is enabled before execution.
- **Persistent Action State:** Finalized action plans are stored in SQLite using deterministic `action_id` values such as `SIM-{date}-{keyword}` . Existing records are updated during reruns to avoid duplicate ticket creation.

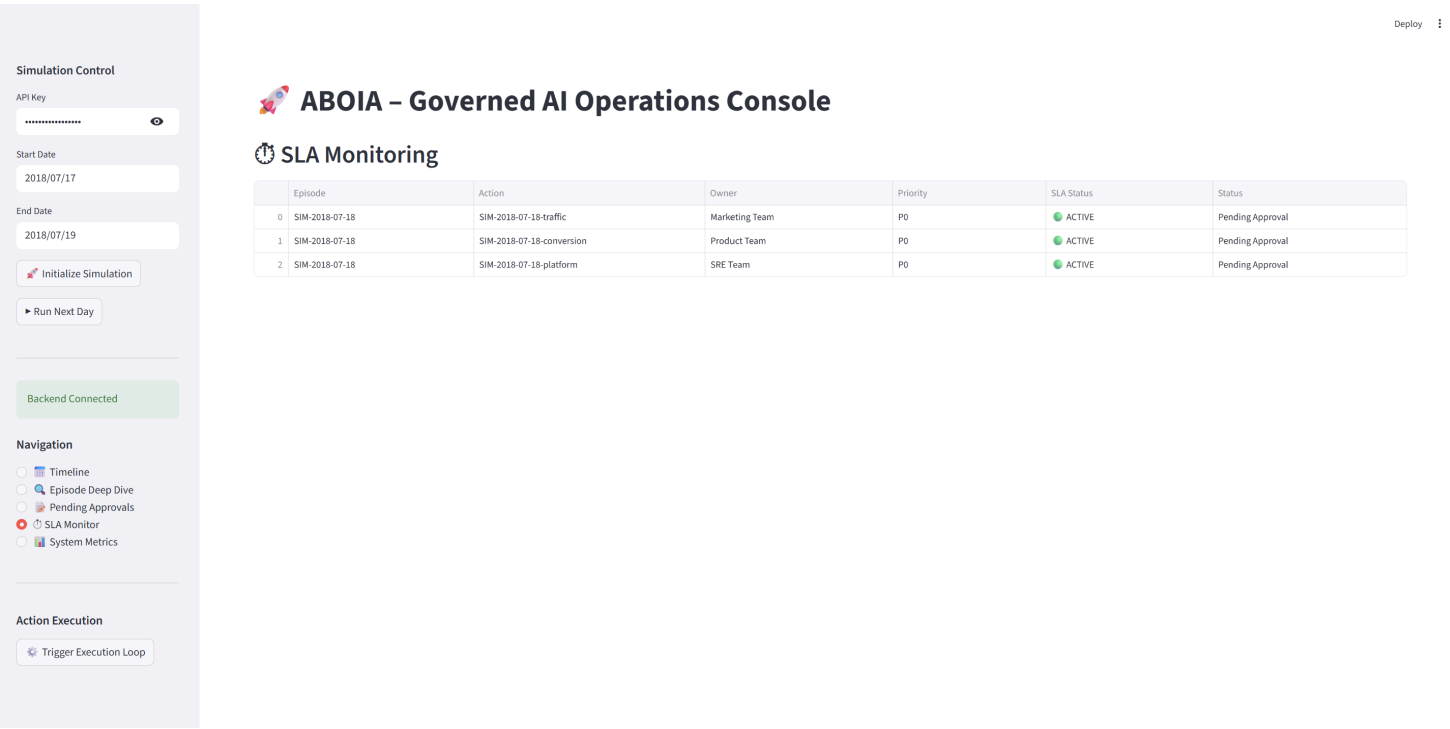
4 Human-in-the-Loop Execution & SLA Tracking (`ActionAgent`)

The `ActionAgent` manages execution workflows and SLA monitoring independently from the LLM reasoning pipeline.

- **Approval-Gated Execution:** Tickets requiring manual approval remain in `pending_approval` until an operator approves or rejects them through the dashboard.
- **Lifecycle Worker:** A background worker continuously polls approved tickets from SQLite, claims execution ownership using a Compare-And-Swap (CAS) lock, executes the task, and updates the lifecycle state to `completed`.
- **SLA Monitoring:** Each ticket receives an `sla_deadline` based on its priority level. The lifecycle worker audits unresolved tickets during each polling cycle and automatically marks overdue actions as `breached`, triggering escalation alerts to `#aboia_alerts`.





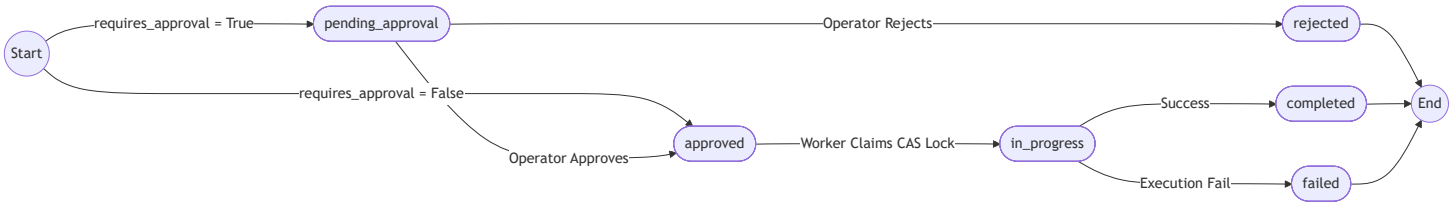


🔄 Action Lifecycle & SLA State Machines

To keep execution predictable and avoid duplicate task processing, the system maintains two independent state machines: one for ticket execution and another for SLA tracking.

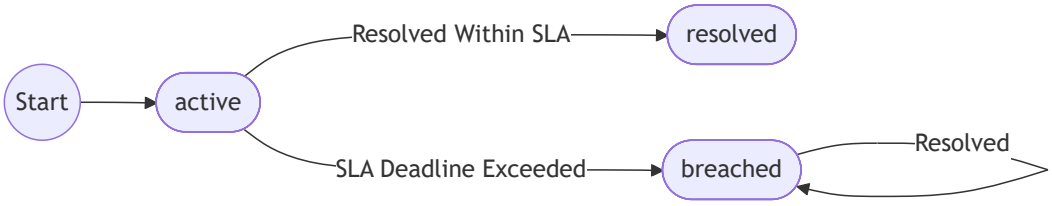
1. Ticket Execution State Machine

Manages approval flow, execution ownership, and task completion states:



2. SLA Compliance State Machine

Tracks SLA deadlines independently from execution state. SLA timers begin when a ticket is created and continue until the ticket is resolved.



📋 Consolidated Pipeline Data Flow Map

The following table summarizes the primary inputs, outputs, and responsibilities of each pipeline stage.

Step / Node	Agent / Component	Inputs	Primary Outputs / Artifacts	Key Responsibility
0️⃣	**DataIngestionAgent**	Raw CSV transaction logs in `data/`	`daily_kpis` (Pandas DataFrame)	Loads and aggregates KPI data while enforcing timeline isolation and warmup rules.
1️⃣	**MonitoringAgent**	`daily_kpis` + historical KPI data	`metrics_df` + anomaly episodes	Runs statistical anomaly detection, seasonal analysis, and cross-metric correlation checks.
2️⃣	**ReasoningAgent**	`anomalies` + `metrics_df`	LLM reasoning output + governance scorecard	Generates structured root-cause reasoning and validates the response using governance checks.
3️⃣	**PlannerAgent**	Reasoning output + Validation results	Persisted action_plan records	Maps diagnoses to predefined operational actions and applies governance override rules.
4️⃣	**ActionAgent**	`action_plan`	Ticket updates + Slack notifications	Handles approvals, execution lifecycle management, and SLA monitoring.

📐 Agent Data Contracts

All agents communicate using structured JSON contracts. Each pipeline stage receives a well-defined input and produces a predictable output, allowing components to evolve independently without breaking downstream workflows.

1. Monitoring Output — Anomaly Record (`anomalies`)

A deduplicated anomaly event generated by the statistical monitoring engine when a KPI breaches its threshold.


```
{
  "date": "2024-01-15",
  "metric": "visits",
  "value": 1823.0,
  "type": "zscore",
  "score": 3.4
}
```


2. Reasoning Output — LLM Result (`reasoning`)

Structured reasoning payload returned by Gemini after analyzing the anomaly summary.

```
{
  "root_cause": "Traffic drop from paid campaigns",
  "business_impact": "Revenue at risk due to reduced funnel entry",
  "risk_level": "high",
  "anomaly_confidence": 90,
  "reasoning_confidence": 85
}
```


3. Governance Validation Output (`validation`)

Deterministic governance evaluation generated by the `ReasoningValidator` .


```
{
  "is_valid": true,
  "severity": "none",
  "issues": [],
  "evaluation": {
    "confidence_gap_score": 95,
    "grounding_score": 100,
    "risk_alignment_score": 100,
    "analytical_depth_score": 90,
    "overall_reasoning_score": 96,
    "explanations": {
      "risk_alignment": "Perfect alignment. The generated risk level 'high' appropriately matches the structural episode severi
      "confidence_gap": "Acceptable confidence variance. AI certainty (85%) is reasonably close to the statistical anomaly stre
      "grounding": "Excellent metric grounding. The reasoning explicitly references the affected database metrics: ['visits']."
      "analytical_depth": "High analytical depth achieved. Detected advanced reasoning signals: percentage_change, baseline_ref
    }
  }
}
```


4. Planner Output- Action Plan (`action_plan`)

Final action plan persisted after governance evaluation and action mapping.

```
{
  "generated_at": "2024-01-15T10:00:00.000000",
  "episode_id": "ep_20240115_visits",
  "risk_level": "high",
  "priority": "P0",
  "actions": [
    {
      "action_id": "ep_20240115_visits-traffic",
      "type": "investigation",
      "description": "Investigate traffic sources and ad campaigns",
      "owner": "Marketing Team",
      "priority": "P0",
      "sla_hours": 4,
      "requires_approval": true,
      "approval_role": "Business Head",
      "status": "pending"
    }
  ],
  "decision_trace": {
    "episode_id": "ep_20240115_visits",
    "risk_level": "high",
    "anomaly_confidence": 90,
    "reasoning_confidence": 85,
    "validation_severity": "none",
    "confidence_gap": 5,
    "evaluation_scores": {
      "confidence_gap_score": 95,
      "grounding_score": 100,
      "risk_alignment_score": 100,
      "analytical_depth_score": 90,
      "overall_reasoning_score": 96,
      "explanations": {
        "risk_alignment": "Perfect alignment. The generated risk level 'high' appropriately matches the structural episode seve
        "confidence_gap": "Acceptable confidence variance. AI certainty (85%) is reasonably close to the statistical anomaly st
        "grounding": "Excellent metric grounding. The reasoning explicitly references the affected database metrics: ['visits']
        "analytical_depth": "High analytical depth achieved. Detected advanced reasoning signals: percentage_change, baseline_r
      }
    },
    "escalation_flags": []
  },
  "validation": {
    "is_valid": true,
    "severity": "none",
    "issues": []
  }
}
```


⚖️ Mathematical Governance & Risk Policy

ABOIA uses deterministic governance rules to validate LLM-generated reasoning before operational actions are created or executed.

1. Risk-to-Priority Policy

Validated incident severity is mapped to operational priority levels, SLA targets, and approval requirements.

Risk Level	Priority	Target SLA	Requires Approval
`high`	**P0**	4 Hours	✅ Always (Business Head)
`medium`	**P1**	8 Hours	❌ Default No
`low`	**P2**	24 Hours	❌ Default No

2. Confidence Alignment Validation

The system compares two independent confidence signals to detect mismatches between statistical anomaly strength and LLM reasoning confidence.

- `anomaly_confidence` **(Deterministic, 0–100):** Computed from anomaly severity, metric overlap, and the number of affected KPIs.

$$\textbf{anomaly\_confidence} = \min(\text{Metric Impact} + \text{Overlap Bonus} + \text{Severity Bonus}, 100)$$

  Where:
  - **Metric Impact:** $\min(N \times 15, 45)$, where N is the number of affected metrics
  - **Overlap Bonus:** $+25$ for overlapping anomalies
  - **Severity Bonus:** $+20$ for `high` severity and $+10$ for `medium`
- `reasoning_confidence` **(Probabilistic, 0–100):** Returned directly by Gemini as part of the structured reasoning response.

If the confidence gap exceeds governance thresholds, the planner escalates the ticket priority and routes execution through manual approval.

> [!NOTE:] Governance thresholds and scoring weights are externalized through `risk_policy.yaml`, allowing policy adjustments without modifying application code.

3. Governance Escalation & Priority Overrides

When governance checks fail or confidence signals become unreliable, the `PlannerAgent` applies override rules that escalate ticket priority and require manual approval before execution.

Escalation Override Rule	Trigger Condition	Execution Penalty / Action
Critical Validation Override	`validation_severity == "critical"`	Priority escalated by one level + `requires_approval = True`
Confidence Gap Override	`abs(anomaly_confidence - reasoning_confidence) > 50`	`requires_approval = True`
Unrecognized Playbook Override	No matching action mapping found	Priority escalated by one level + `requires_approval = True`
Low Overall Reasoning Override	`overall_reasoning_score < 50`	Priority escalated by one level + `requires_approval = True`
Weak Metric Grounding Override	`grounding_score < 40`	`requires_approval = True`

Escalation Override Rule	Trigger Condition	Execution Penalty / Action
Risk Misalignment Override	`risk_alignment_score < 60`	Priority escalated by one level + `requires_approval = True`

⚙️ Configuration-Driven Orchestration

To keep operational logic separate from application code, KPI definitions, monitoring rules, governance policies, and action mappings are externalized through YAML configuration files.

This allows system behavior to be updated without modifying the core Python services.

- `kpis.yaml` : Defines KPI aggregation rules, source columns, and derived metric formulas such as `conversion_rate = orders / visits` .
- `monitoring.yaml` : Specifies the list of metrics monitored by the anomaly detection pipeline.
- `risk_policy.yaml` : Stores risk-to-priority mappings, SLA targets, approval requirements, and escalation policies.
- `root_cause_action_map.yaml` : Maps LLM-generated root-cause keywords to predefined operational actions, ticket types, and owner teams.

🧠 Engineering Design Decisions

State Management & Data Flow (LangGraph)

The pipeline is orchestrated using a shared `LangGraph StateGraph` , where each node receives an `AgentState` object, updates it, and passes the modified state to the next stage.

Core state fields include:

- KPI datasets,
- anomaly records,
- governance validation results,
- reasoning outputs,
- action plans,
- and execution lifecycle status.

Agents remain stateless between runs, with all execution context flowing through the shared state object.

`episode_id` values are generated dynamically from the simulated date (`SIM-{simulated_date}`) to keep execution deterministic across reruns and simulation resets.

Although the current workflow is sequential, LangGraph also provides future support for checkpoint recovery, approval interrupts, and non-linear execution paths without restructuring the orchestration layer.

Architectural Decision: Synchronous Pipeline vs. Asynchronous Execution

ABOIA uses a hybrid execution model where the AI pipeline runs synchronously while ticket execution runs independently.

- **Synchronous Pipeline:** All agents execute sequentially inside the LangGraph workflow. Each stage completes before the next begins, ensuring deterministic state flow and chronological isolation during simulation runs.
- **Asynchronous Ticket Execution:** After the `PlannerAgent` persists the action plan to SQLite, the pipeline terminates. Ticket approvals, execution, and SLA auditing are handled separately through the Streamlit UI and the background lifecycle worker (`/v1/system/run_lifecycle`).

Observability & Debugging Traceability

Each pipeline stage writes structured debug artifacts to `/debug_output` , making it possible to trace agent inputs, outputs, and LLM interactions throughout execution.

- `llm_calls.ndjson` : Raw prompts, responses, model metadata, and execution timings.
- `node_state.ndjson` : `AgentState` snapshots for each LangGraph node transition.
- `reasoning_validation.ndjson` : Governance evaluation scores and validation explanations.
- `planner_agent.ndjson` : Action mappings, escalation flags, and approval decisions.
- `anomalies_full.json` & `anomalies_current_day.json` : Statistical anomaly outputs for current and historical monitoring windows.
- **Streamlit Payload Inspector**: The dashboard also exposes expandable raw API payload traces directly in the UI for easier debugging and inspection.

LLM Reliability & Safety Design

The `llm_service.py` module treats the Gemini API as an external dependency and includes several safeguards for reliability and failure handling.

- **Timeout Handling:** Configurable request timeouts (default: 60 seconds) prevent stalled API calls from blocking the pipeline.
- **Retry Logic:** Failed requests are retried automatically using exponential backoff (`2 ** attempt`) with up to three total attempts.
- **Structured JSON Parsing:** Responses are parsed using regex-based JSON extraction to isolate structured payloads from conversational text formatting.
- **Fallback Responses:** If the LLM repeatedly fails or returns invalid JSON, the system falls back to a predefined reasoning payload instead of terminating the workflow.

```
{
  "root_cause": "LLM unavailable after retries",
  "business_impact": "Automated reasoning could not be generated.",
  "risk_level": "medium",
  "anomaly_confidence": 75,
  "reasoning_confidence": 0
}
```

- **Prompt Constraints:** The model is instructed to reason only from the provided anomaly summary and avoid unsupported assumptions or external events.
- **Structured Context Compilation:** The `ReasoningAgent` converts KPI statistics, z-scores, rolling averages, and metric deltas into a structured summary before invoking the LLM.

Database State & Auditing (`SQLite + SQLAlchemy`)

Pipeline state is persisted through SQLAlchemy ORM models defined in `app/db/models.py` .

- **`Episode` Model:** Stores anomaly episodes, generated reasoning output, and the associated action plan payload.
- **`Action` Model:** Stores operational ticket state, including:
  - execution lifecycle status,
  - approval state,
  - SLA deadlines,

    - and breach tracking fields.
- **Lifecycle & SLA Fields**
    - Key execution states include:
        - `pending_approval`
        - `approved`
        - `in_progress`
        - `completed`
        - `rejected`
        - `failed`
    - SLA tracking fields include:
        - `sla_hours`
        - `sla_deadline`
        - `sla_status`
- **Compare-And-Swap (CAS) Execution Lock**The lifecycle worker transitions tickets into `in_progress` only if the current database state is still `approved` .

```
rows_updated = db.query(Action).filter(
    Action.action_id == action_record.action_id,
    Action.status == "approved"
).update({"status": "in_progress"})
```

  If no rows are updated, execution is skipped to prevent duplicate processing.
  Although SQLite serializes writes in the current simulation environment, the CAS pattern was implemented to support future migration to distributed database systems such as PostgreSQL.
- **Episode-to-Action Linking:** `Episode` and `Action` records are linked through deterministic ID patterns (`SIM-{date}-{keyword}`) rather than strict foreign-key constraints, allowing both models to be queried independently during simulation runs.

REST API Reference

The FastAPI backend exposes REST endpoints used by the Streamlit dashboard and external automation workflows.

Interactive API documentation is available through:

- Swagger UI: `http://127.0.0.1:8000/docs`
- ReDoc: `http://127.0.0.1:8000/redoc`

1. Pipeline Execution

Method	Endpoint	Description
`POST`	`/v1/run_simulation`	Runs the full simulation pipeline for a given date range. (developer utility endpoint; not used by the Streamlit UI dashboard).
`POST`	`/v1/run_day`	Runs the pipeline for a single simulation day.(primary endpoint driving the Streamlit UI dashboard).

- **Request JSON Payload Schema (for both execution endpoints):**

```
{
  "start_date": "YYYY-MM-DD",
  "end_date": "YYYY-MM-DD"
}
```

2. Action & Ticket Management

Method	Endpoint	Description
GET	/v1/actions/	Returns historical action records with optional filtering.
GET	/v1/actions/{action_id}	Returns detailed information for a specific action ticket.

- **Example Queries:**

```
GET /v1/actions/?status=pending_approval
GET /v1/actions/?priority=P0&owner=Marketing Team
GET /v1/actions/?sla_status=breached
```

3. Human-in-the-Loop Approvals

Method	Endpoint	Description
GET	/v1/approvals/pending	Returns tickets waiting for manual approval.
POST	/v1/approvals/{action_id}/approve	Approves an action ticket.
POST	/v1/approvals/{action_id}/reject	Rejects an action ticket.

4. Incident Episodes

Method	Endpoint	Description
GET	/v1/episodes/	Returns anomaly episodes, reasoning summaries, and governance results.

5. SLA Monitoring

Method	Endpoint	Description
GET	/v1/sla/breached	Returns tickets that exceeded their SLA deadline.
GET	/v1/sla/active	Returns currently active SLA-tracked tickets.

6. System & Lifecycle Operations

Method	Endpoint	Description
GET	/	Basic server status endpoint.
GET	/v1/system/health	Returns application health information.
GET	/v1/system/metrics	Returns aggregated system metrics and ticket statistics.
GET	/v1/system/kpi_window	Returns KPI history for dashboard visualization.
POST	/v1/system/run_lifecycle	Runs the lifecycle worker for approvals, execution, and SLA auditing.

📈 API Versioning & Pagination

- All API routes are grouped under the `/v1` prefix to support future API versioning.
- Paginated endpoints such as `/v1/actions/` and `/v1/episodes/` support `limit` and `offset` query parameters to control response size.

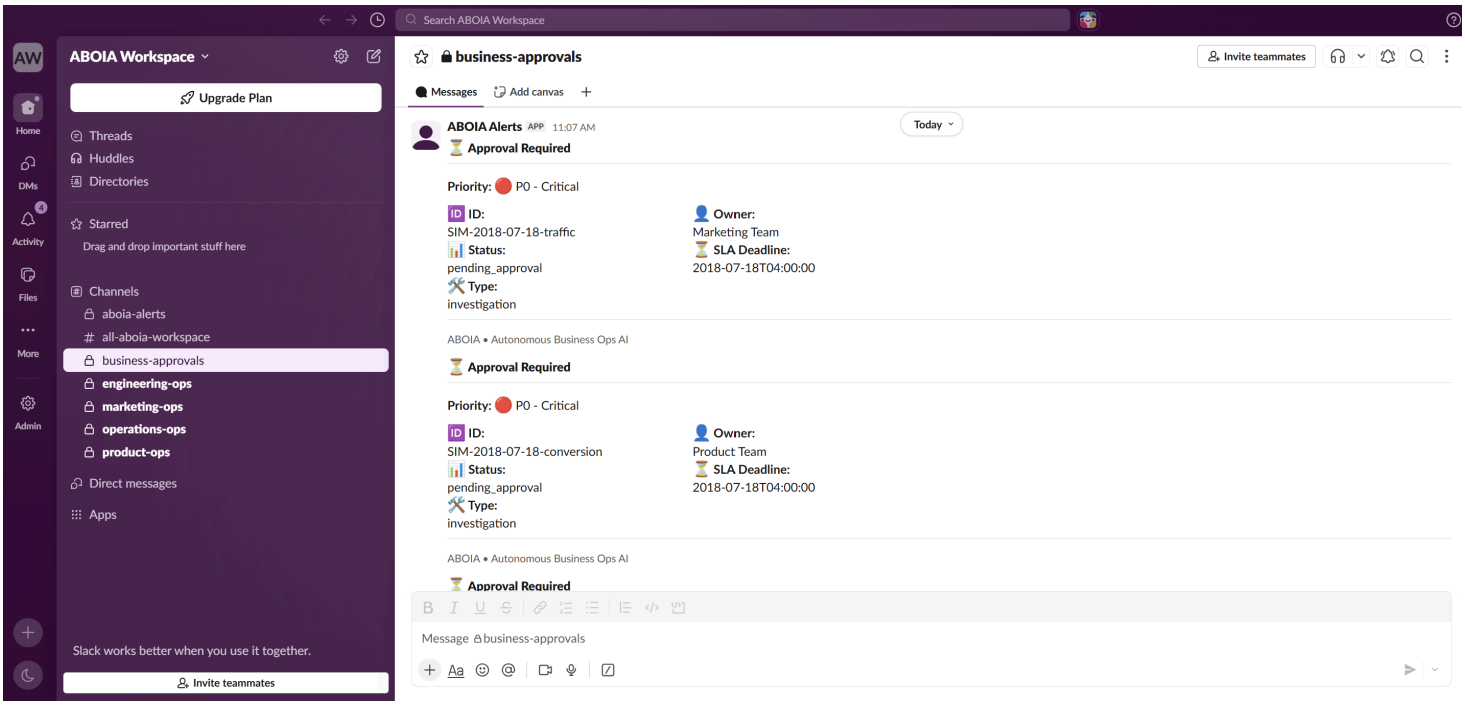
🛡️ API Authentication

Endpoints that modify system state or trigger execution require an `x-api-key` header. Read-only monitoring and observability endpoints remain public.

- **Secured Endpoints (Require `x-api-key` HTTP Header):**
  - `POST /v1/run_simulation`
  - `POST /v1/run_day`
  - `POST /v1/system/run_lifecycle`
  - `POST /v1/approvals/{action_id}/approve`
  - `POST /v1/approvals/{action_id}/reject`
- **Public Observability Endpoints:**
  - `GET /`
  - `GET /v1/system/health`
  - `GET /v1/system/metrics`
  - `GET /v1/system/kpi_window`
  - `GET /v1/actions/`
  - `GET /v1/actions/{action_id}`
  - `GET /v1/episodes/`
  - `GET /v1/sla/breached`
  - `GET /v1/sla/active`
  - `GET /v1/approvals/pending`

🔔 Slack Notification Routing

The `NotificationService` sends Slack notifications whenever a ticket changes state. Alerts are routed dynamically based on the ticket owner and event type. For example, marketing-related actions are sent to `#marketing_ops`, while SLA breaches are routed to `#aboia_alerts`.



Slack Event Routing Matrix

Event	Destination Channel	Notification Details
`created`	Owner Team Channel (e.g., `#marketing_ops`)	Action ID, Owner team, and Priority Indicator (P0/P1/P2) using a lightweight block header UI.
`approval_required`	Owner Team Channel + Centralized `#approvals`	Standard Action details (ID, owner, status, target SLA deadline, type) and priority indicator, prompting for manual operator action.
`approved`	Owner Team Channel + Centralized `#approvals`	Standard Action details (ID, owner, status, target SLA deadline, type) and priority indicator, confirming human-in-the-loop authorization.
`rejected`	Owner Team Channel + Centralized `#approvals`	Standard Action details (ID, owner, status, target SLA deadline, type) and priority indicator, confirming operator cancellation.
`completed`	Action Owner's Team Channel	Standard Action details (ID, owner, status, target SLA deadline, type) and priority indicator, confirming successful system execution.
`failed`	Action Owner's Team Channel	Standard Action details (ID, owner, status, target SLA deadline, type), priority indicator, and the precise execution error message.
`sla_breached`	`#aboia_alerts`	Standard Action details (ID, owner, status, target SLA deadline, type) and priority indicator, highlighting breached virtual calendar deadlines.

🛡️ Operational Scenarios Demonstrated

ABOIA supports three primary execution flows based on governance validation results and incident severity:

1. **Automatic Execution Flow (Happy Path):**
   If the `ReasoningValidator` returns acceptable governance scores and the incident risk level is low or medium, the system automatically creates and executes the action ticket without requiring manual approval.

2. **Governance Override & Manual Approval (Override Path):**
   If the governance score falls below the approval threshold, or if a large confidence mismatch is detected between statistical signals and LLM reasoning, the `PlannerAgent` escalates the ticket priority and places the action in `pending_approval` until manually reviewed through the dashboard.
3. **SLA Breach Handling (Temporal Overruns):**
   If an unresolved ticket exceeds its configured SLA window, the lifecycle worker automatically marks the ticket as `breached` and dispatches a high-priority alert to `#aboia_alerts` .

🛠️ Technology Stack

- **Orchestration:** `LangGraph`
- **LLM/Reasoning:** `Gemini 2.5 Flash`
- **Backend API:** `FastAPI` , `Uvicorn` , `Pydantic`
- **Frontend Dashboard:** `Streamlit` , `Pandas`
- **Persistence Layer:** `SQLite` , `SQLAlchemy`
- **Notifications:** Slack Web API (`chat.postMessage`)
- **Language:** `Python`

📊 Simulation Results: A 4-Day Chronological Simulation Walkthrough

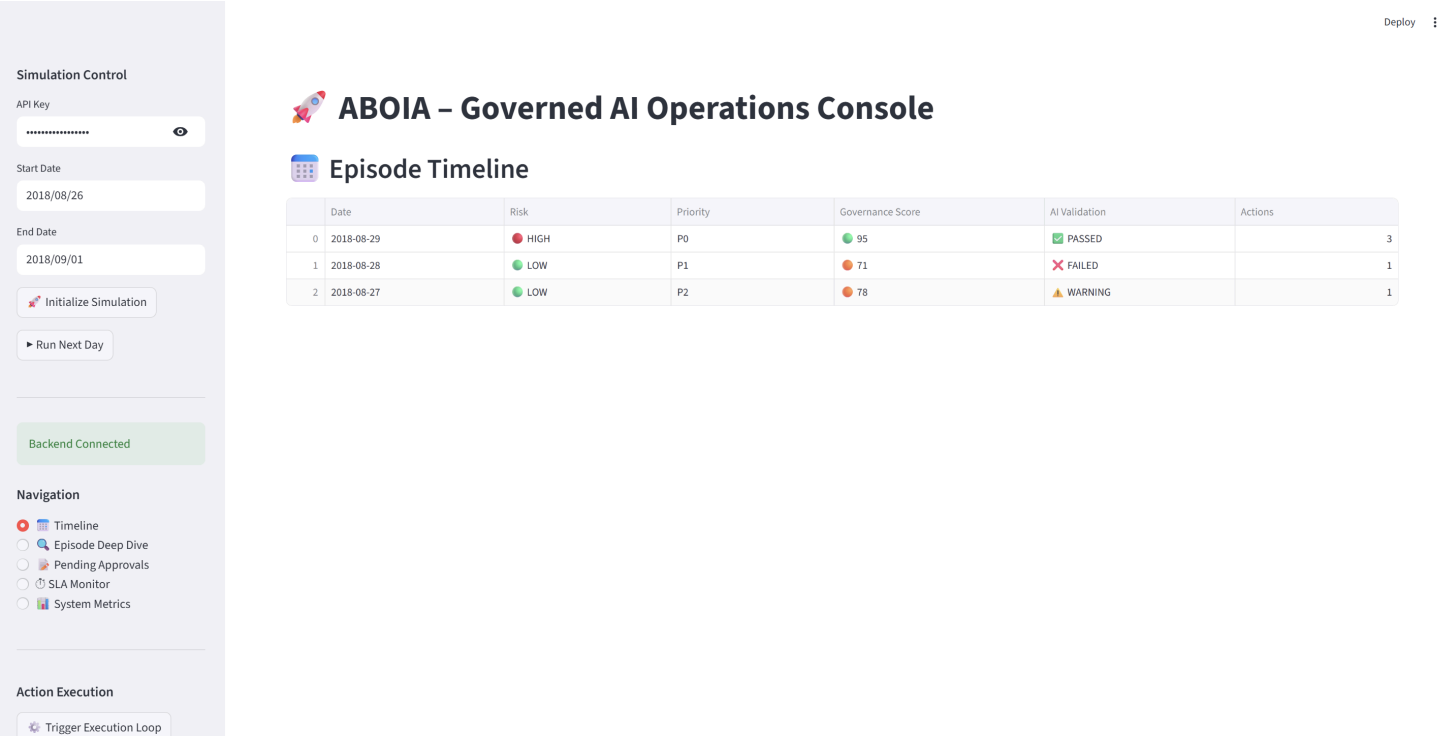
The system was evaluated across four consecutive simulation days (`2018-08-26` to `2018-08-29`) using the Olist e-commerce dataset.

This timeline demonstrates:

- a stable no-anomaly day,
- a low-severity anomaly with automatic execution,
- a governance override requiring manual approval,
- and a high-severity multi-metric incident escalated to a P0 workflow.

📸 Consolidated Episode Timeline Dashboard

The dashboard below shows the complete operational timeline across all simulated days.



📋 Chronological Simulation Run Summary

Day / Date	Monitoring Outcome	Severity	LLM Diagnosis	Governance Score	Planner Decision	Execution Outcome
Day 1 2018-08-26	All KPIs within baseline ranges (< 1.5\sigma).	None	*No reasoning step triggered.*	N/A	No action created requires_approval = False	No tickets, approvals, or notifications generated.
Day 2 2018-08-27	aov showed a seasonal deviation (-2.1σ , value: 92.60)	Low	Temporary shift in purchasing behavior affecting AOV.	**Overall: 78/100** *Grounding: 100* *Parity: 50*	P2 Ticket Created requires_approval = False	Automatically executed and resolved. Slack notification sent to #marketing_ops
Day 3 2018-08-28	gmv showed a seasonal deviation (-2.3σ , value: 4121.22).	Low	Reduction in daily GMV.	**Overall: 71/100** *Grounding: 100* *Parity: 15*	Escalated from P2 to P1 due to confidence mismatch. requires_approval = True	Ticket locked in pending_approval until manual review.
Day 4 2018-08-29	Seven concurrent anomalies detected, including orders (-4.6σ) and conversion_rate (-5.1σ).	High	Potential checkout or platform infrastructure outage.	**Overall: 95/100** *Grounding: 100* *Parity: 95*	P0 Incident Created requires_approval = True	High-priority approval workflow triggered with Slack escalation to #aboia_alerts .

🔍 In-Depth Chronological Operational Case Studies

🟢 Day 1: Quiet Operational Stability (August 26, 2018)

All KPIs remained within normal baseline ranges (variances below `1.5σ`). The `MonitoringAgent` detected no anomalies, so the pipeline terminated after the monitoring stage without invoking the LLM or generating operational tickets. This avoids unnecessary alerts and reduces inference usage during stable periods.
📸 **Visual Evidence:** Day 1 Stable Status.

🟡 Day 2: Auto-Approved Remediation (August 27, 2018)

The `MonitoringAgent` detected a seasonal `aov` deviation (`92.60`, `-2.1σ`). Gemini identified the issue as a temporary shift in purchasing behavior and returned an overall governance score of `78/100`.

Although the validator detected a confidence mismatch warning, the score remained above the approval threshold. The `PlannerAgent` generated a `P2` Marketing Ops ticket with a 24-hour SLA, and the lifecycle worker automatically executed the task to `completed`.
📸 **Visual Evidence:** Funnel Outlier Plots | AI Reasoning Analysis | Governance Scorecard | Action Plan Execution.

🟠 Day 3: Governance Override & Manual Approval (August 28, 2018)

The `MonitoringAgent` flagged a seasonal `gmv` drop (`4,121.22`, `-2.3σ`). Gemini generated a high-confidence diagnosis, but the statistical anomaly score remained low, creating a large confidence gap between deterministic and LLM-generated signals.

The `ReasoningValidator` triggered a `confidence_gap_override`, escalating the ticket priority from `P2` to `P1` and setting `requires_approval = True`. The action remained locked in `pending_approval` until manual review.
📸 **Visual Evidence:** Funnel Outlier Plots | AI Reasoning Deep Dive | Governance Evaluation | Action Plan Approvals Lock.

🔴 Day 4: Multi-Metric Outage & P0 Escalation (August 29, 2018)

The monitoring ensemble detected seven concurrent anomalies, including severe drops in `conversion_rate` (`-5.1σ`) and `orders` (`-4.6σ`). Gemini identified the event as a potential payment or checkout system failure and returned a governance score of `95/100`.

The `PlannerAgent` escalated the incident to `P0` severity with a 4-hour SLA and generated three concurrent remediation tasks covering failover handling, operational alerts, and infrastructure response workflows.

All actions remained in `pending_approval` until manually approved through the HITL workflow.
📸 **Visual Evidence:** Funnel Outlier Plot 1 & Plot 2 | AI Reasoning Deep Dive | Governance Scorecard |Action Plan Approvals Lock.

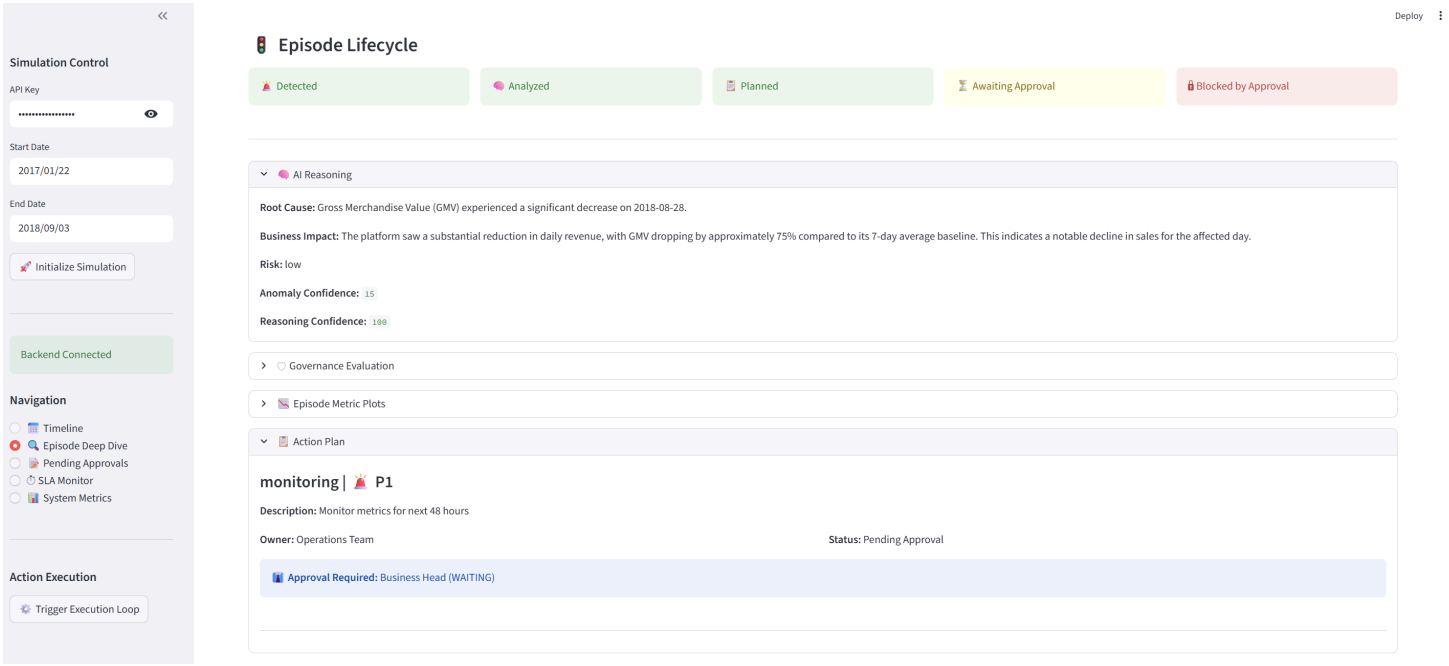
🔄 Operational Lifecycle Validation: Approval Gates & SLA Compliance

The following walkthrough demonstrates three operational flows within the system: standard approval execution, manual rejection, and SLA breach handling.

1️⃣ Human-in-the-Loop: Approval Lifecycle

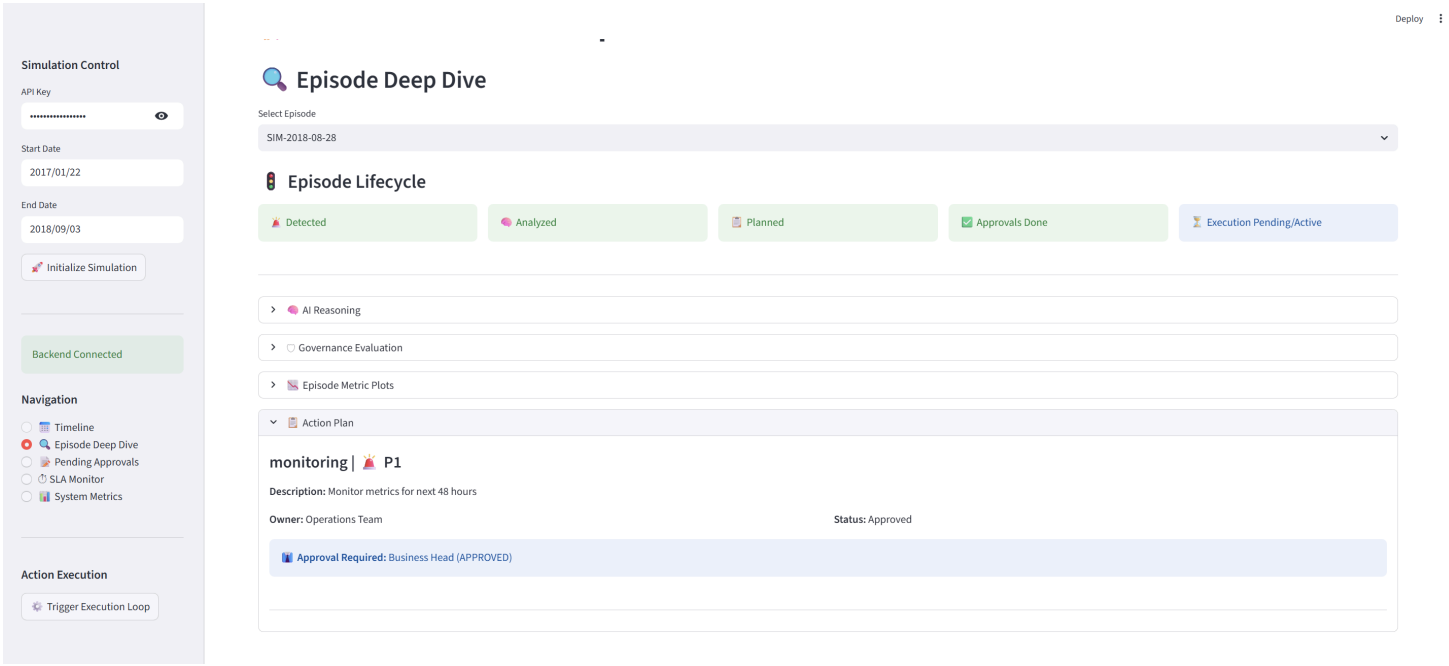
The August 28th GMV anomaly triggered a governance override due to a large confidence mismatch between the statistical anomaly score and the LLM reasoning confidence. As a result, the generated ticket was placed in `pending_approval`.

Step A- Pending Approval
The ticket remains locked until an operator approves it through the dashboard.



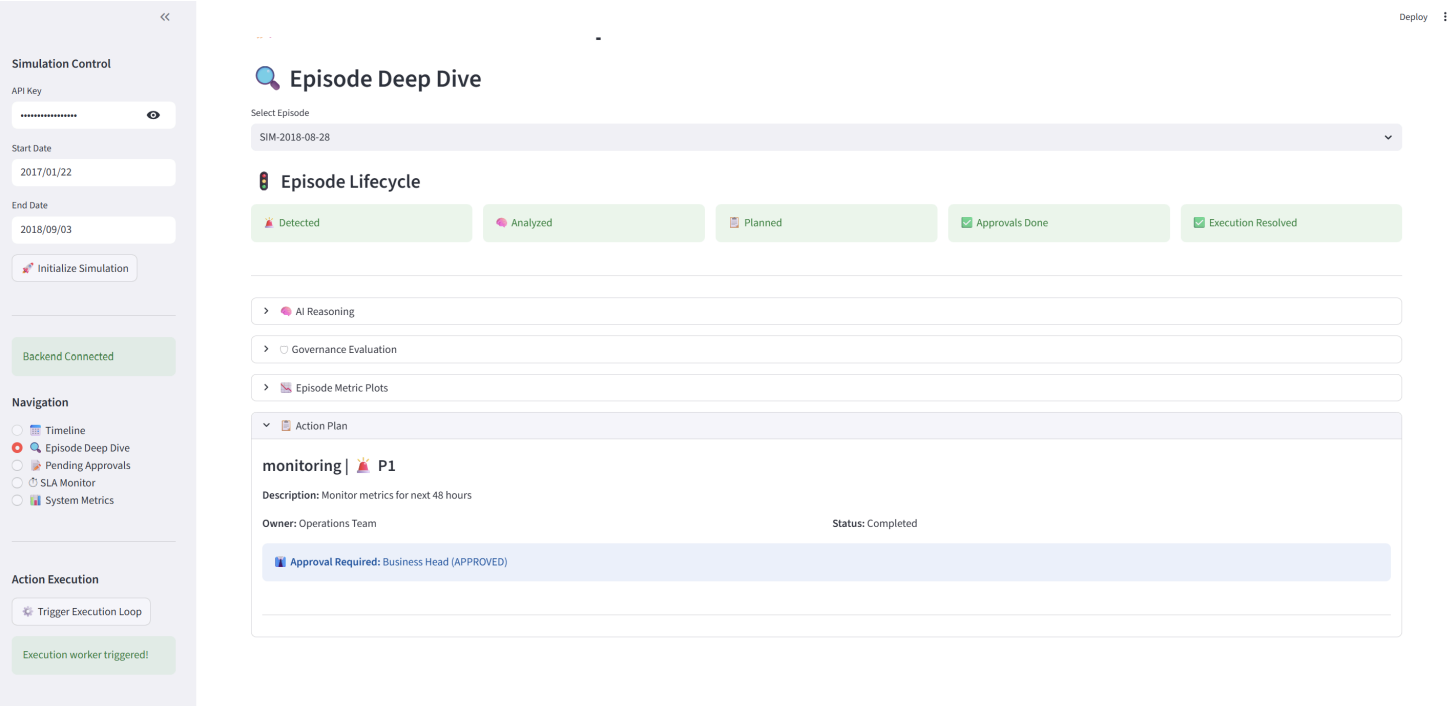
Step B- Approved State

After approval, the ticket is removed from the pending approvals queue and marked as `approved`.



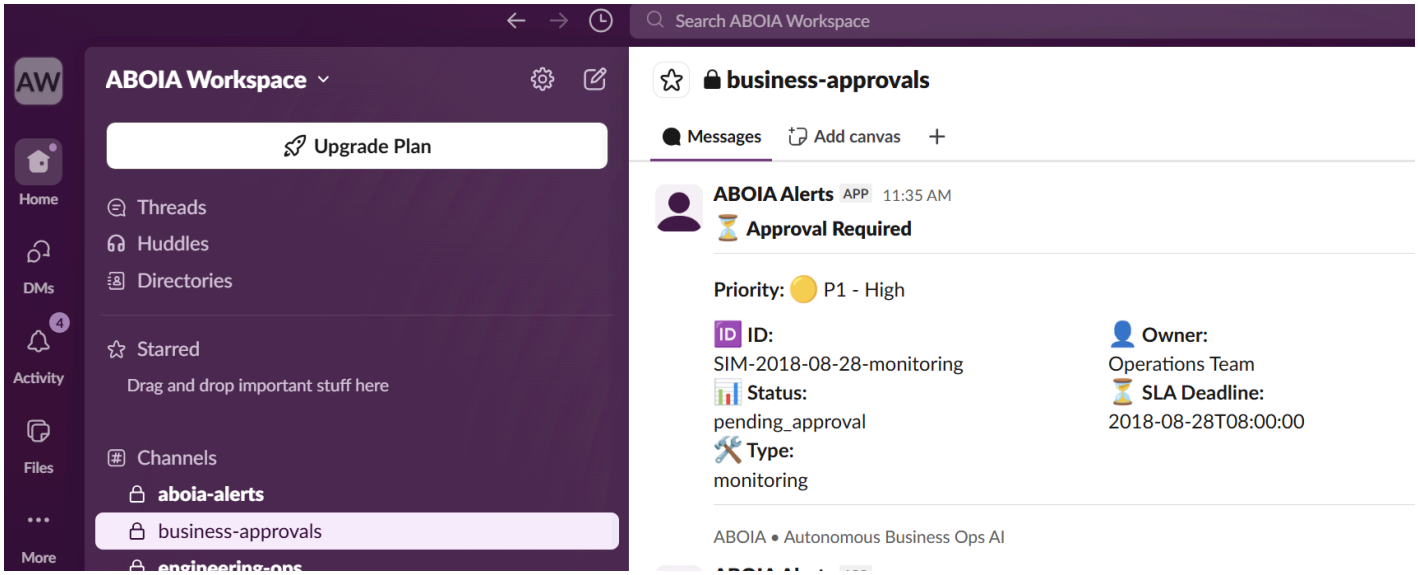
Step C- Execution Lifecycle

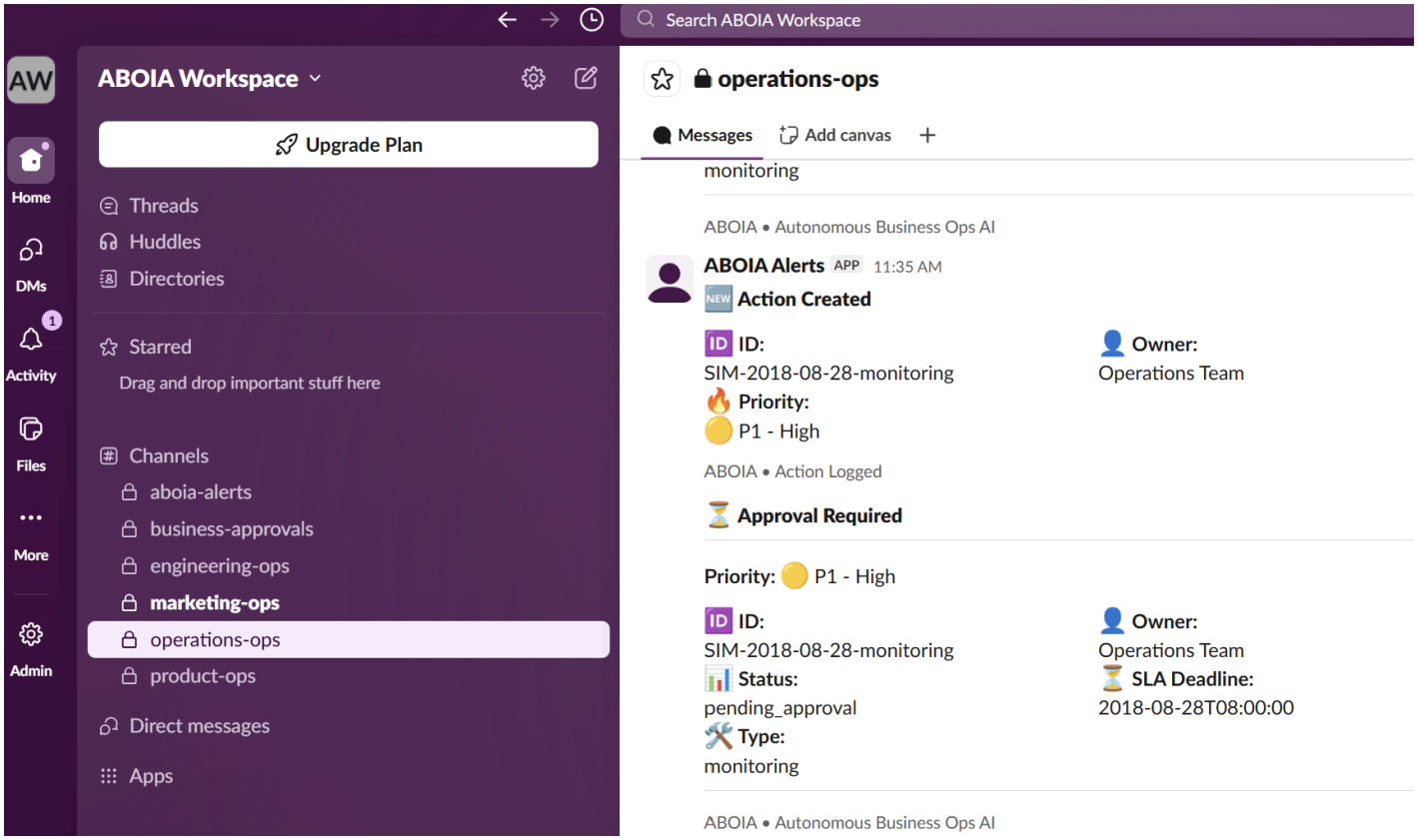
When the execution worker runs, it claims the ticket lock, transitions the status to `in_progress`, performs the simulated execution step, and finally marks the action as `completed`.



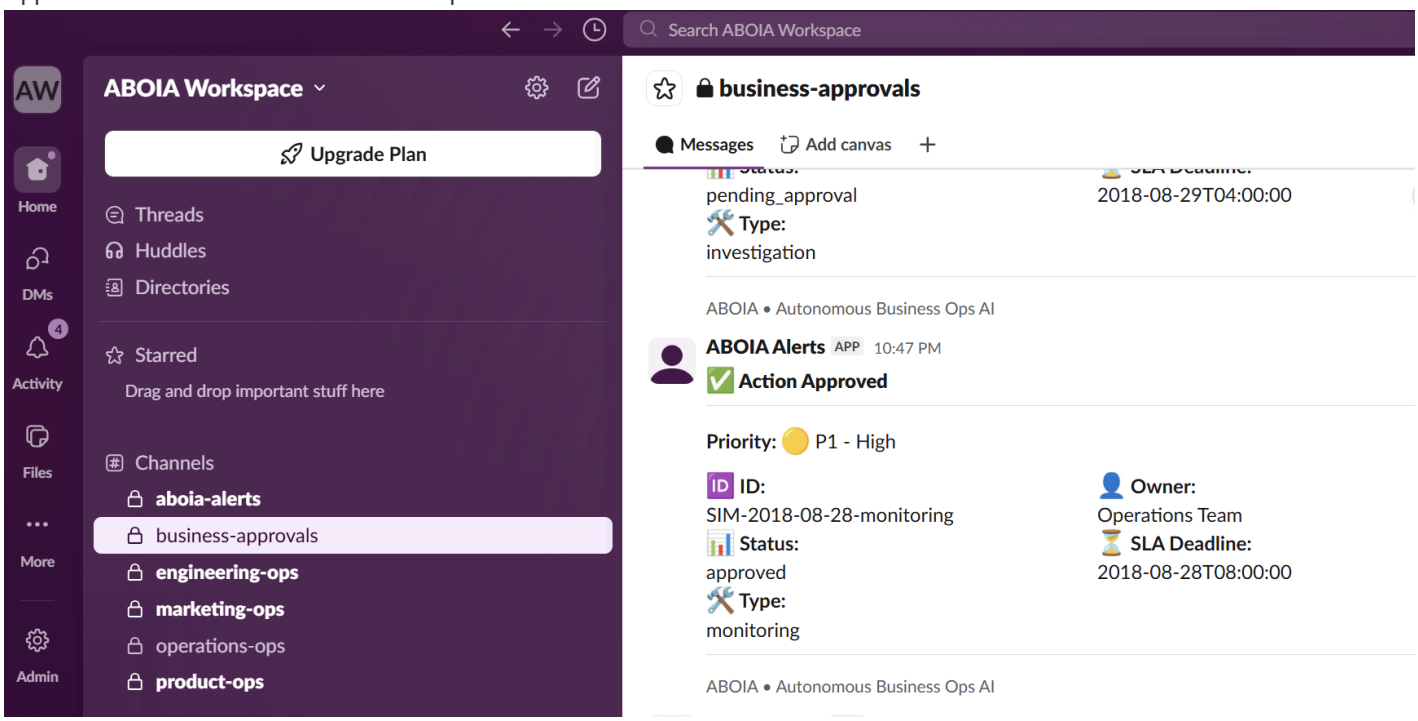
Slack Notifications:

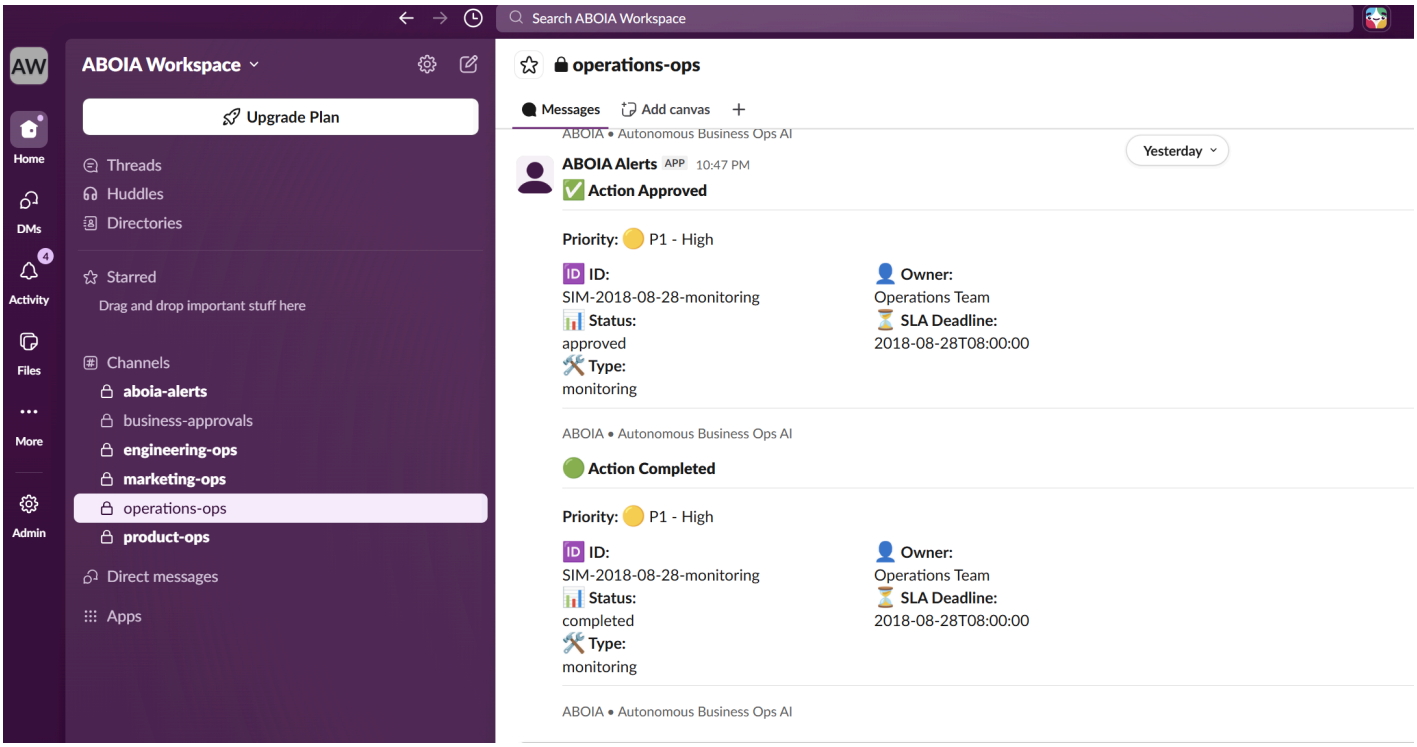
- Approval request and action creation alerts:





- Approval confirmation and execution completion alerts:





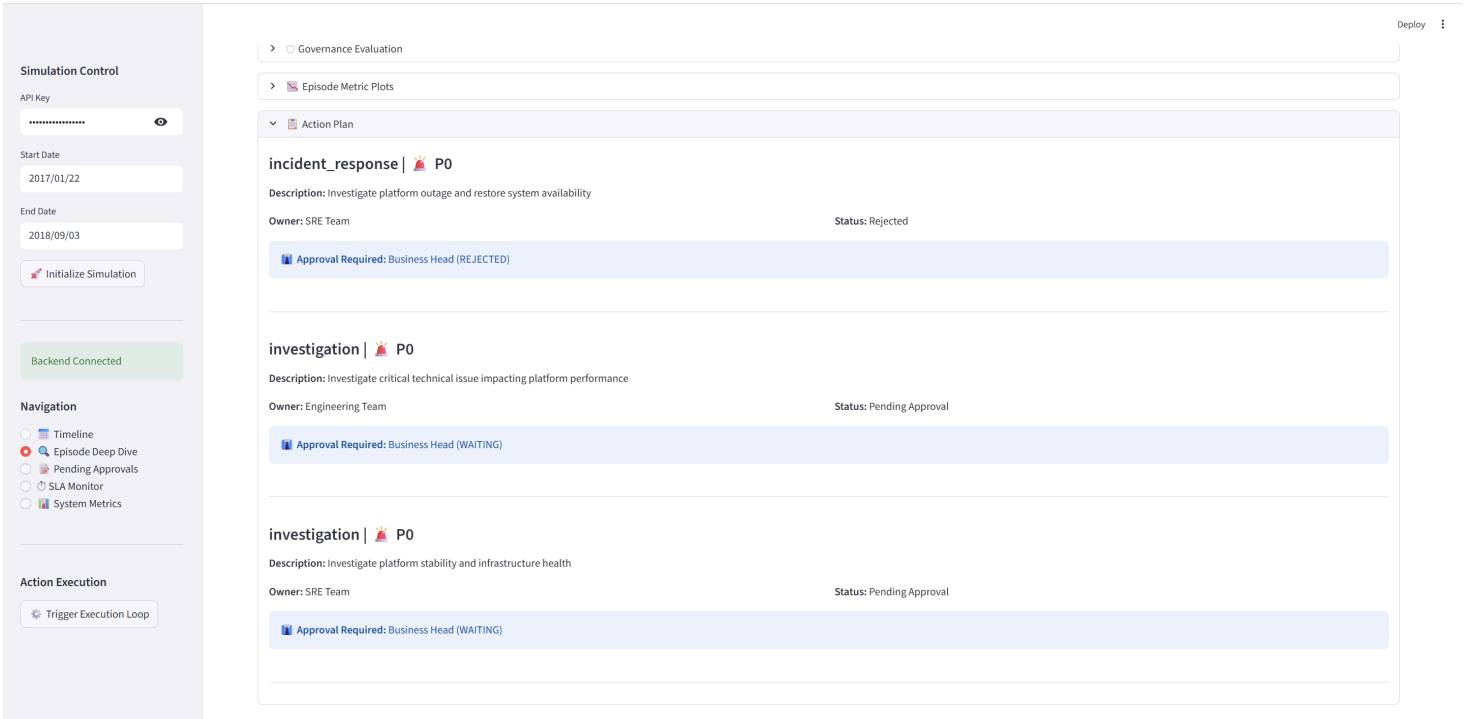
2️⃣ Human-in-the-Loop: Rejection Lifecycle

On the August 29th P0 outage (Day 4), the `PlannerAgent` generated three concurrent action tickets. One of the tickets was manually rejected from the Pending Approvals queue.

Clicking **Reject** transitions the ticket state from `pending_approval` to `rejected`, removes it from the active approvals queue, and marks the SLA state as `resolved`. Rejected tickets are therefore excluded from the active SLA Monitor view.

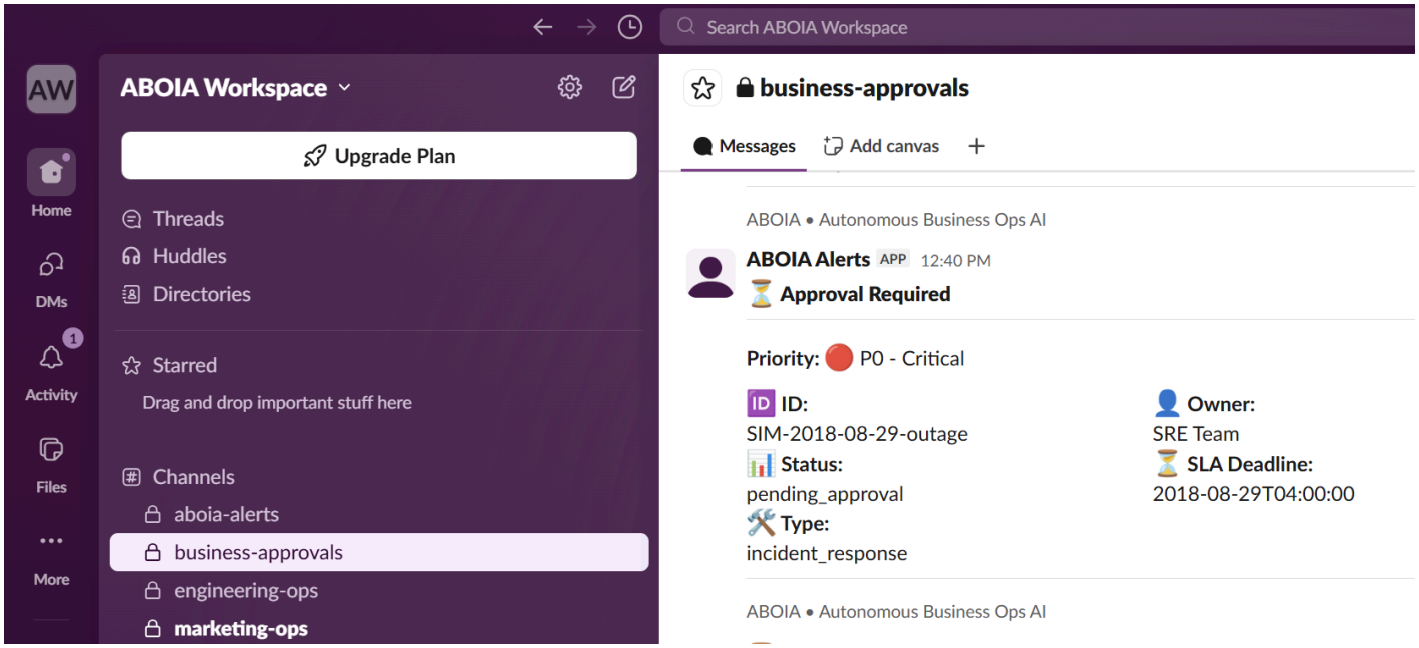
The lifecycle worker only processes tickets with `status == "approved"`, so rejected actions are never executed. Under the current multi-action workflow, rejecting any required action prevents the overall episode from being fully resolved, avoiding partial execution across dependent tasks.

Visual Evidence: The rejected status is persisted in the episode history and displayed within the action plan panel:

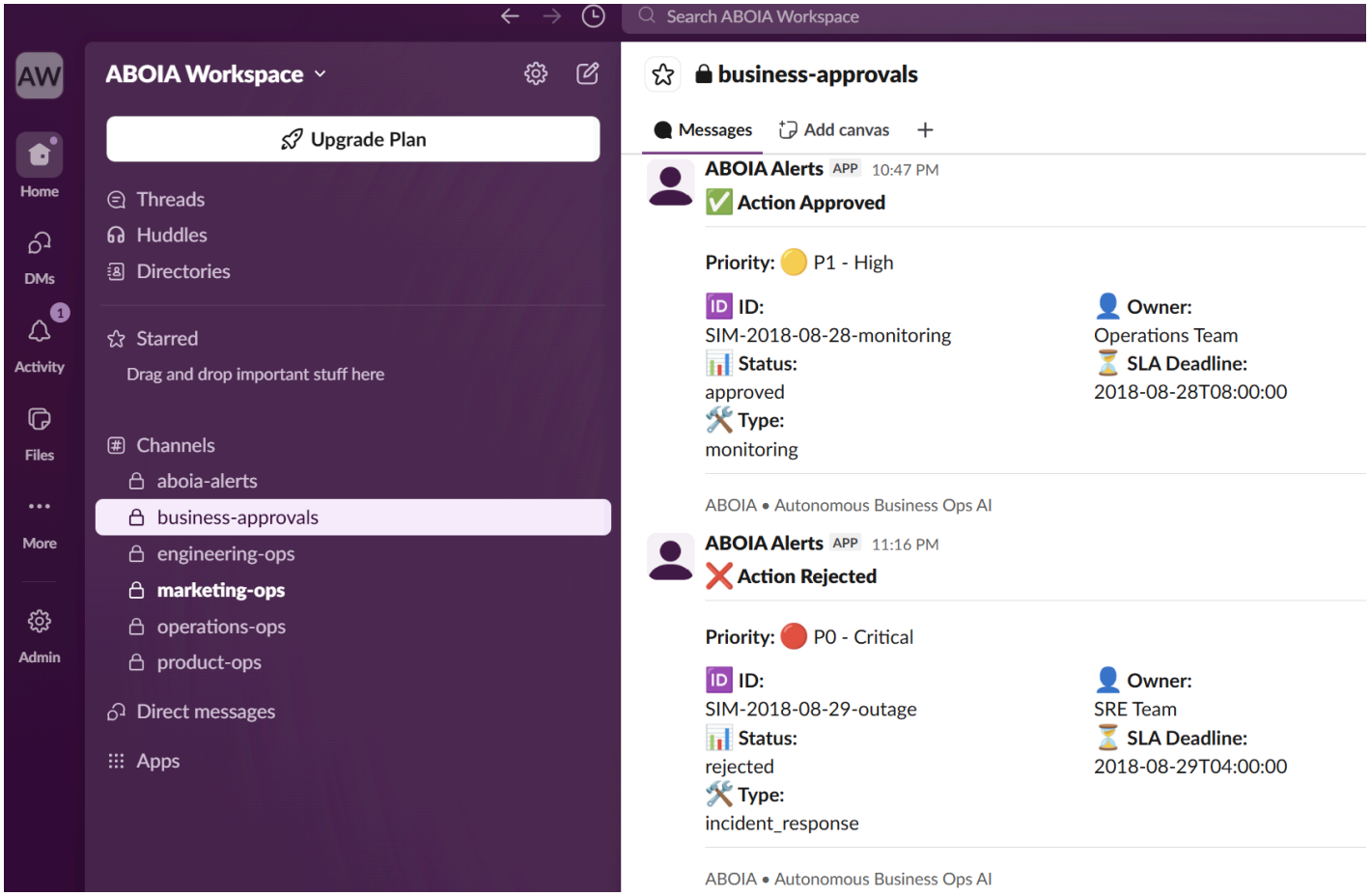


Slack Notifications:

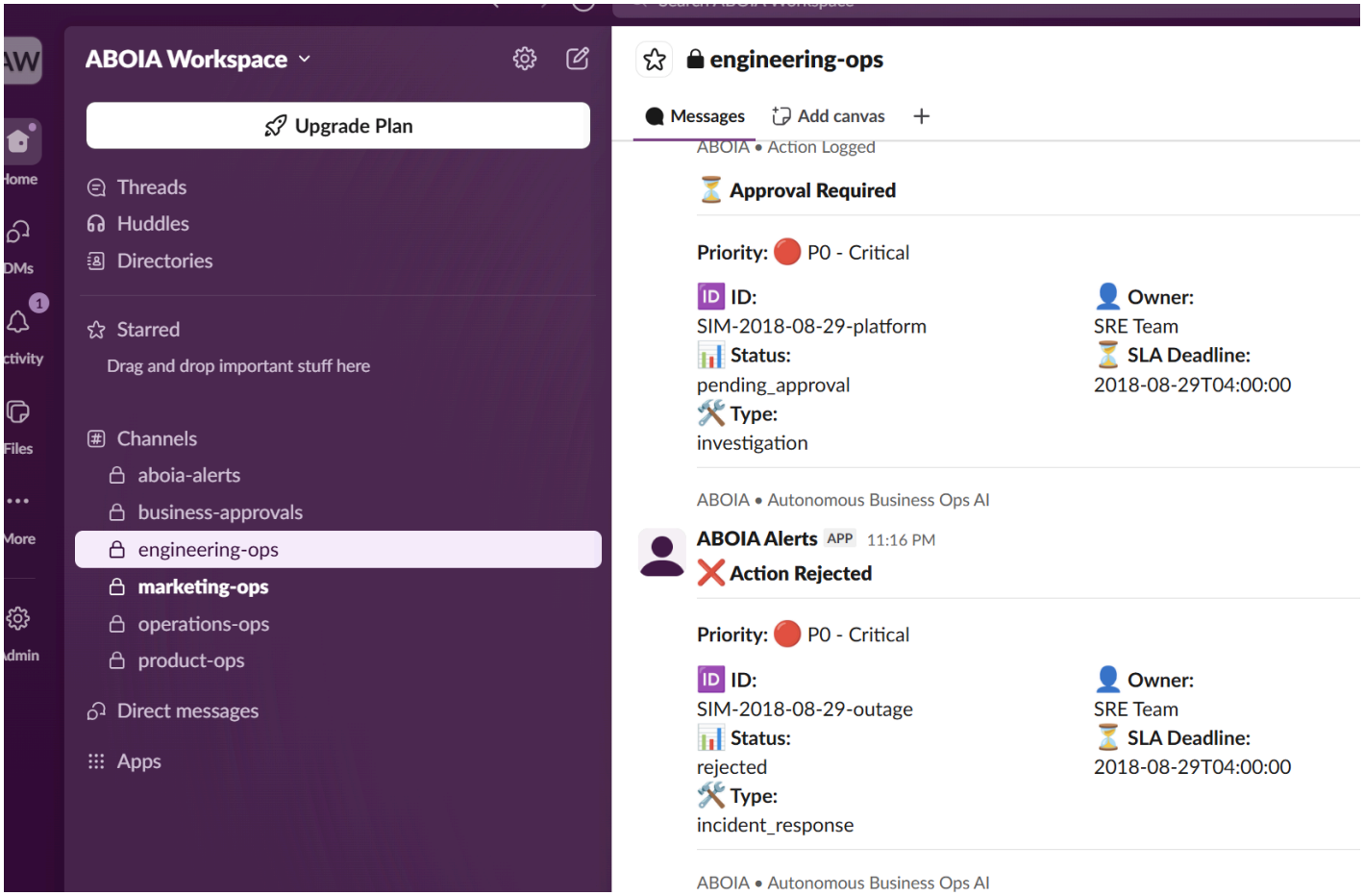
- Pending approval notification in `#business-approvals`:



- Rejection confirmation sent to `#business-approvals`:



- Engineering team notification sent to `#engineering-ops`:

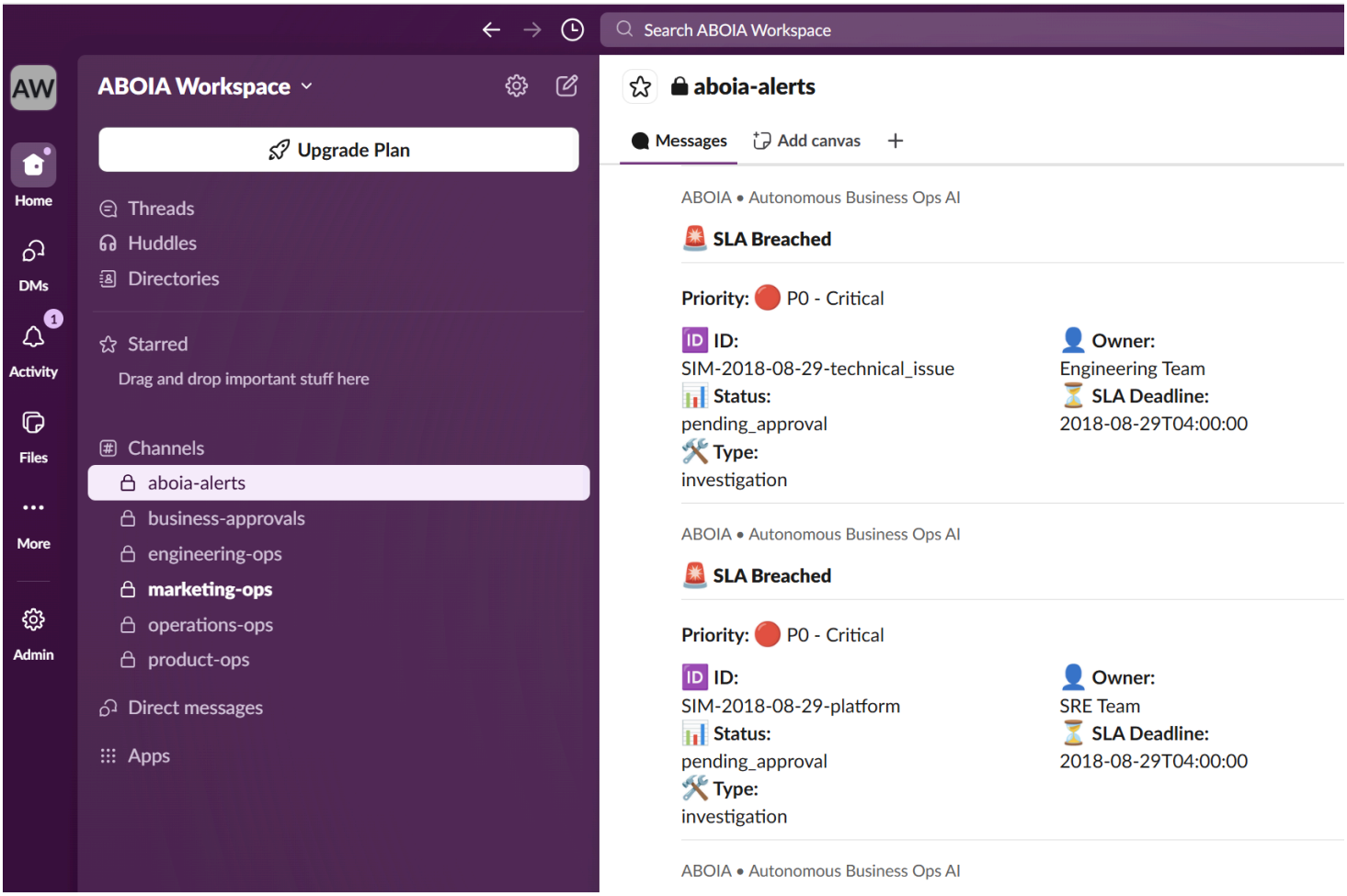


3️⃣ Simulation-Aware SLA Compliance Tracking

The SLA engine tracks deadlines against simulated calendar time rather than wall-clock execution time. Only unresolved tickets appear in the SLA Monitor table, while completed and rejected tasks are removed automatically.

On August 27th, the P2 action resolved immediately and never entered SLA monitoring. On August 29th, the P0 incident remained in `pending_approval` beyond its 4-hour SLA window. During the next lifecycle poll, the worker updated the SLA state from `active` to `breached`.

A breached SLA also triggers a high-priority Slack alert routed to `#aboia_alerts`.



💻 Developer Setup & Installation

Prerequisites

- Python 3.10+
- Google Gemini API Key
- *(Optional)* Slack Bot Token

1. Clone & Install

```
git clone https://github.com/your-username/ABOIA.git
cd ABOIA

python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

2. Configure Environment Variables

Create a local `.env` file from the template:

```
cp .env.example .env
```

Update the `.env` file with your API keys and optional Slack configuration:

```
# Application API Key
API_KEY="your_secure_api_key"

# Gemini API Key
GEMINI_API_KEY="your_gemini_api_key"

# Optional Model Override
GEMINI_MODEL="gemini-2.5-flash"

# Optional Slack Integration
SLACK_BOT_TOKEN="xoxb-your-bot-token"

SLACK_MARKETING_CHANNEL="#marketing_ops"
SLACK_PRODUCT_CHANNEL="#product_ops"
SLACK_ENGINEERING_CHANNEL="#engineering_ops"
SLACK_OPERATIONS_CHANNEL="#business_ops"
SLACK_APPROVAL_CHANNEL="#approvals"
SLACK_ALERT_CHANNEL="#aboia_alerts"
```

> If `SLACK_BOT_TOKEN` is not configured, Slack notifications are skipped automatically without interrupting the simulation workflow.

3. Dataset Setup

The project uses the Olist Brazilian e-commerce dataset for KPI generation and anomaly simulation.
Place the following CSV files inside `data/raw/` :

- `olist_customers_dataset.csv`
- `olist_orders_dataset.csv`
- `olist_order_items_dataset.csv`

The repository already includes the required dataset structure. You can also replace the CSVs with a different transactional dataset if needed.

4. Running the Application

The project consists of:

- a FastAPI backend,
- a Streamlit dashboard,
- and a SQLite persistence layer.

Start the backend and frontend in separate terminals.

Option A: Run the Dashboard

- **Terminal 1 (FastAPI Backend):**

```
uvicorn app.main:app --reload
```

- **Terminal 2 (Streamlit Frontend):**

```
streamlit run streamlit_app/dashboard.py
```

Option B: Run through REST API

You can also trigger simulations directly using REST endpoints.

- **Run a Single Simulation Day:**

```
curl -X POST http://127.0.0.1:8000/v1/run_day \
  -H "x-api-key: your_secure_api_key" \
  -H "Content-Type: application/json" \
  -d '{"start_date": "2024-01-15", "end_date": "2024-01-15"}'
```

- **Trigger Lifecycle Worker:**

```
curl -X POST http://127.0.0.1:8000/v1/system/run_lifecycle \
  -H "x-api-key: your_secure_api_key"
```

🔮 Limitations & Future Work (Scaling to Production)

ABOIA currently operates as a local simulation environment focused on anomaly detection, governance validation, and approval-driven execution workflows. While the current architecture is sufficient for controlled simulations, several changes would be required to support large-scale real-time production workloads.

Phase 1: Data Ingestion & Telemetry

1. Real-Time Metric Ingestion

Current Implementation: The `DataIngestionAgent` currently processes static CSV datasets using Pandas. KPI baselines are computed from historical data loaded into memory during simulation runtime.

Future Improvements: For production-scale workloads, the ingestion layer would need to support continuous event streams instead of batch CSV processing. This could include integrating streaming platforms such as Apache Kafka or AWS Kinesis, storing telemetry in time-series databases like ClickHouse or TimescaleDB, and limiting historical queries to rolling windows (for example, the last 60–90 days) to keep memory usage and query latency predictable as data volume grows.

This would allow the monitoring pipeline to compute rolling baselines, seasonal trends, and anomaly statistics continuously in near real time.

2. Data Quality & Missing Telemetry

Current Implementation: The ingestion layer validates KPI schemas before processing. Missing numeric values are repaired using interpolation and forward/backward filling to avoid pipeline interruptions, while structural issues such as missing columns or invalid dates immediately stop execution.

Future Improvements: In production, ingestion failures should generate dedicated data-quality incidents instead of terminating the workflow. Additional safeguards such as bounded outlier handling would also help prevent corrupted telemetry from distorting anomaly baselines.

Phase 2: Statistical Monitoring & Anomaly Detection

3. Configurable Causal Rules Engine

Current Implementation: The `MonitoringAgent` currently evaluates hardcoded e-commerce metric relationships such as visits → orders → GMV to reduce false-positive correlations. Because these rules are defined directly in Python, the monitoring logic remains

domain-specific.

Future Improvements: Future versions could move these causal rules into configurable YAML-based definitions, allowing the framework to support different business domains without modifying application code.

4. Intra-Day Event Clustering

Current Implementation: The system currently operates on daily simulation windows, where all anomalies for a given date are grouped into a single episode. This can merge unrelated incidents occurring on the same day into one reasoning context.

Future Improvements: Production-scale monitoring would benefit from smaller real-time monitoring windows and event clustering techniques such as DBSCAN to separate unrelated anomalies into independent micro-episodes for more accurate reasoning and remediation.

Phase 3: Cognitive Reasoning & Governance

5. Resilient LLM Routing

Current Implementation: The reasoning pipeline currently depends on a single Gemini API integration, making the system vulnerable to rate limits and upstream outages.

Future Improvements: Production deployments would benefit from a multi-model routing layer with automatic fallback support across providers such as Gemini, Claude, GPT-4o, or self-hosted models. Additional safeguards such as token-budget limits could also help prevent runaway inference costs during large-scale operations.

6. Weighted Governance Scoring

Current Implementation: The overall reasoning score is currently calculated using a simple average across all evaluation dimensions. This gives equal importance to safety-critical checks such as Metric Grounding and Risk Alignment, and secondary factors such as Analytical Depth.

Future Improvements: Future versions could use weighted governance scoring where safety-oriented metrics contribute more heavily to approval decisions. Moving these weights into configuration files would also allow governance policies to be adjusted without modifying application code.

Phase 4: Planning & Task Routing

7. Semantic Routing Improvements

Current Implementation: The `PlannerAgent` currently maps LLM-generated diagnoses to ticket actions using keyword matching defined in `root_cause_action_map.yaml`. While lightweight and deterministic, this approach can miss unexpected LLM phrasing outside the configured synonym list.

Future Improvements: Future versions could improve routing accuracy using embedding-based semantic matching and stricter structured LLM outputs. This would allow the planner to classify diagnoses more reliably without depending entirely on keyword parsing.

8. Closed-Loop Operational Feedback

Current Implementation: The system currently operates in an open-loop workflow. Once a ticket is completed, no feedback is sent back into the monitoring or reasoning pipeline to evaluate whether the remediation actually resolved the anomaly.

Future Improvements: Future iterations could introduce feedback-based learning by linking completed actions with post-resolution KPI behavior. This could help the system prioritize historically effective remediations and dynamically tune anomaly thresholds based on operational outcomes over time.

Phase 5: Execution & Human-in-the-Loop Operations

9. External Integrations & Webhook Execution

Current Implementation: The `ActionAgent` currently simulates ticket execution locally through SQLite state updates.

Future Improvements: Production deployments would require integration with external systems such as Jira, PagerDuty, Shopify, or Stripe through authenticated APIs and event-driven webhooks for real operational execution.

10. Multi-Channel Alerting & Fallback Handling

Current Implementation: Notifications currently rely on Slack integration. If Slack is unavailable or unconfigured, alerts are skipped without interrupting the workflow.

Future Improvements: A production alerting layer would require fallback notification channels such as PagerDuty, email, or SMS to ensure critical incidents are still delivered during outages. External heartbeat monitoring could also help detect failures in the orchestration system itself.

11. Hardened Human-in-the-Loop (HITL) Security Controls

Current Implementation: Approval actions are currently triggered directly through the Streamlit interface without operator authentication or identity tracking.

Future Improvements: Production systems would require stronger approval controls such as signed approval tokens, operator identity auditing, and multi-step approval flows for high-risk actions.

Phase 6: Infrastructure & Security

12. Distributed Asynchronous Task Queueing & Workers

Current Implementation: The pipeline currently executes sequentially to preserve simulation consistency and simplify state management.

Future Improvements: Production deployments would benefit from asynchronous task queues using systems such as Celery or Temporal to process LLM workflows outside the main API thread. This would improve scalability, retry handling, and overall system responsiveness.

13. Policy-as-Code Governance

Current Implementation: Governance thresholds and validation metrics — such as grounding scores, risk alignment checks, and reasoning score weights — are currently configured through `config/risk_policy.yaml`.

Future Improvements: For larger distributed deployments, these policies could be moved into a centralized Policy-as-Code layer using frameworks such as Open Policy Agent (OPA) or Rego, enabling cross-service policy management and compliance updates without modifying application code.

14. Scalable Persistence Layer

Current Implementation: The project currently uses SQLite for local persistence, which is sufficient for simulation workloads but limited for concurrent multi-user environments.

Future Improvements: Production systems would require a scalable database layer such as PostgreSQL with connection pooling, read replicas, and stronger concurrency support for distributed deployments.

15. Enterprise-Grade Authentication & RBAC

Current Implementation: API access is currently protected using a static `x-api-key`.

Future Improvements: Enterprise deployments would require OAuth/OIDC-based authentication and role-based access control (RBAC) to separate permissions across viewers, operators, and governance administrators.

16. Distributed Observability & Tracing

Current Implementation: The system currently relies on structured debug logs and NDJSON traces for local observability.

Future Improvements: Production-scale monitoring would benefit from distributed tracing and telemetry platforms such as OpenTelemetry, Jaeger, or Datadog for end-to-end visibility across services and workflows.